

Research paper

A resilient IoT intrusion detection system using hybrid feature selection and explainable ensemble learning

Abubakar Wakili ^{*}, Sara Bakkali

Euromed University of Fes, Fes, Morocco

ARTICLE INFO

Keywords:

Internet of Things (IoT) security
Intrusion detection systems (IDS)
Ensemble learning
Hybrid feature selection
Explainable artificial intelligence (XAI)
Fault-tolerant systems
Machine learning for cybersecurity
Resilient digital infrastructure

ABSTRACT

Internet of Things (IoT) networks, comprising billions of connected devices, have become prime targets for cyberattacks that threaten critical digital infrastructure. Traditional intrusion detection systems often fail when individual components encounter implementation barriers, creating security coverage gaps in operational environments. This paper presents a fault-tolerant ensemble framework for multi-class IoT intrusion detection that maintains operational continuity despite component failures. The framework integrates a hybrid feature selection approach that combines statistical filters, recursive feature elimination, and embedded tree-based ranking, achieving a 27.7% dimensionality reduction (from 47 to 34 features). An adaptive ensemble architecture incorporating five models (Balanced Random Forest, LightGBM, CatBoost, Extra Trees, and XGBoost) is designed to handle realistic implementation challenges, with systematic fault tolerance when components fail. When evaluated on the CICIoT2023 dataset, the framework achieved 99.16% accuracy, 73.79% macro F1-score, and 0.9908 MCC while demonstrating systematic fault tolerance. Experimental validation for explainability revealed that packet Inter-Arrival Time, TCP protocol features, and traffic magnitude are the most discriminative attack indicators. These insights help security teams interpret and trust detection outcomes. By integrating high detection performance, fault tolerance, and interpretability, the proposed approach provides a practical pathway for resilient IoT cybersecurity systems and strengthens the foundations for sustainable digital infrastructure protection.

1. Introduction

Securing distributed computing infrastructures has become a critical priority for organizations and governments. Traditional perimeter-based defenses, originally designed for static enterprise networks, are poorly suited to today's dynamic, heterogeneous, and highly interconnected environments [1]. The Internet of Things (IoT) has intensified challenges by introducing billions of resource-constrained devices across sectors such as healthcare, manufacturing, energy, transportation, and smart cities [2,3]. These devices often operate with limited CPU, memory, and power, rely on proprietary protocols, and communicate in mobile or ad hoc topologies, significantly expanding the attack surface and complicating threat detection [4,5].

The societal and economic stakes are substantial. In healthcare, compromised infusion pumps or patient monitors could disrupt care and jeopardize safety; in manufacturing, tampered sensors could halt pro-

duction lines and cause costly downtime; in smart cities, attacks on edge nodes controlling traffic or power systems could endanger public safety. Campaigns such as Mirai and Mozi have demonstrated the scale of IoT exploitation, harnessing hundreds of thousands of devices for distributed denial-of-service (DDoS) attacks and malware propagation [6–8]. Industry analyses estimate billions of dollars in annual losses from operational downtime, data breaches, and reputational damage, highlighting the urgent need for resilient IoT security.

These realities demand intrusion detection systems (IDS) that are accurate, lightweight, explainable, and fault-tolerant. Unlike enterprise security appliances, IoT IDS must work within tight resource budgets while addressing extreme class imbalance, where rare but critical attack types may constitute less than 0.01% of traffic [6]. Device heterogeneity compounds this challenge, requiring models capable of multi-class detection rather than binary classification [9,10]. Explainability is equally important; security teams need to trust and act on alerts. Recent ad-

^{*} Corresponding author.

E-mail addresses: a.wakili@ueuromed.org (A. Wakili), s.bakkali@insa.ueuromed.org (S. Bakkali).

<https://doi.org/10.1016/j.rineng.2025.107392>

Received 2 June 2025; Received in revised form 29 August 2025; Accepted 20 September 2025

vances in explainable AI, applied successfully in telecommunications and healthcare, highlight the value of interpretable models for operational decision-making and regulatory compliance [11,12].

Despite advances, many IDS approaches remain confined to laboratory settings and falter in deployment. Frameworks often assume ideal datasets, neglect integration issues (software conflicts, library versions), and employ deep learning models that are computationally heavy and opaque [13,14]. Memory limitations, model failures, and black-box behavior limit adoption in high-stakes domains. Research shows that ensemble learning with hyperparameter optimization can improve robustness [15,16], and feature selection methods enhance efficiency and interpretability in high-dimensional data [17]. Yet few frameworks integrate these methods with mechanisms for fault tolerance and systematic reliability, which are crucial in healthcare, smart manufacturing, and critical infrastructure [18,19].

This paper addresses these gaps by introducing a resilient, explainable intrusion detection framework for IoT networks, designed with deployment viability in mind. The main contributions are:

- **Fault-Tolerant Hybrid Feature Selection:** Introducing adaptive ensemble scoring with indicator functions that automatically handle method failures, library incompatibilities, and timeout issues. The novel weighted ranking mechanism maintains feature selection consistency when individual methods (statistical, wrapper, embedded) encounter implementation barriers, achieving 27.7% dimensionality reduction (47→34 features) while preserving attack detection capability.
- **Resilient Ensemble Architecture with Automatic Adaptation:** Developing dynamic soft voting that automatically adjusts to component availability when base learners fail during training or encounter compatibility issues. The system maintains ensemble effectiveness with any subset of successfully trained models, demonstrated through continued operation with 4 of 5 intended models.
- **Dual-Tier Explainability Framework:** Implementing systematic fallback between SHAP analysis and permutation importance to ensure interpretable insights remain available despite technical constraints. The approach identifies critical attack indicators (packet timing, TCP protocol behavior, traffic magnitude) while providing guaranteed explainability continuity.
- **Systematic Architectural Framework:** Proposing a fault-tolerant processing pipeline with mathematical foundations for IoT security deployment scenarios, including hybrid cloud-edge considerations for resource-constrained environments.

The framework addresses security challenges spanning critical IoT domains including healthcare device networks, smart city sensor infrastructure, and industrial control systems. Comprehensive evaluation on the CIIoT2023 dataset demonstrates robust performance with 99.16% accuracy, 73.79% macro F1-score, and 0.9908 MCC across 33 attack categories, validating the approach's effectiveness for multi-class threat detection in complex, imbalanced network environments. By integrating accuracy, interpretability, and robustness, this research advances IoT cybersecurity with direct societal and economic benefits, supporting broader goals of cyber resilience and digital trust.

The remainder of this paper is structured as follows. Section 2 reviews related work and highlights key research gaps. Section 3 introduces the proposed methodology, including algorithmic design and mathematical foundations. Section 4 details the experimental setup and implementation environment. Section 5 presents and interprets the results, followed by Section 6, which discusses real-world applicability and deployment considerations. Section 7 provides a comparative analysis against state-of-the-art methods, while Section 8 outlines the limitations and suggests directions for future research. Finally, Section 9 concludes the paper.

2. Related works

2.1. Machine learning approaches in IoT security

Recent advances in IoT intrusion detection have explored various machine learning paradigms to address unique challenges posed by resource-constrained environments and heterogeneous device ecosystems. However, most existing approaches focus primarily on algorithmic innovation while inadequately addressing practical implementation challenges critical for operational deployment.

Ferrag et al. [20] developed a federated deep learning approach using Long Short-Term Memory (LSTM) networks for IoT intrusion detection, achieving 94.12% accuracy on the UNSW-NB15 dataset. While their methodology demonstrated effectiveness in temporal pattern recognition, it was limited to binary classification and did not address extreme class imbalance scenarios or provide guidance for handling implementation failures. Similarly, Authors in [21] proposed a Convolutional Neural Network (CNN) approach achieving 96.4% accuracy on the NSL-KDD dataset, though evaluation did not extend to multi-class IoT-specific attack scenarios or consider practical deployment constraints.

Ensemble learning techniques have gained attention for their ability to improve detection performance through model diversity while providing inherent resilience to individual component failures. Thakkar and Lohiya [22] investigated bagging and boosting ensemble methods for network intrusion detection, achieving 97.8% accuracy through Random Forest implementation. However, their approach lacked comprehensive class imbalance handling and did not provide interpretability mechanisms required for operational deployment. More recently, Punia et al. [23] proposed a machine learning-based anomaly detection system tailored for compromised IoT networks. Also Saheed and Chukwuere [24] introduced an explainable ensemble transfer learning framework for botnet detection in IoT environments. These contributions highlight the trend toward combining efficiency with explainability in IoT security, aligning closely with the objectives of this work.

2.2. Feature selection in cybersecurity applications

Feature selection plays a crucial role in cybersecurity applications where high-dimensional data and noise significantly impact detection performance. Recent research has explored various approaches specifically tailored for cybersecurity datasets, though practical implementation often reveals additional complexity requiring adaptive strategies.

Binbusayyis and Vaiyapuri [25] implemented a comprehensive comparison of feature selection methods for intrusion detection, evaluating filter, wrapper, and embedded approaches across multiple datasets. Their analysis demonstrated that ensemble feature selection methods consistently outperformed individual approaches by 3-5% accuracy improvement, though they did not address implementation stability or error recovery mechanisms essential for operational deployment.

Authors in [26] proposed feature selection techniques for wireless sensor networks using machine learning, focusing on IoT-specific security challenges. While their approach showed promise for IoT environments, evaluation was limited to smaller-scale datasets and did not consider the extreme class imbalance ratios present in comprehensive IoT security datasets.

2.3. Class imbalance challenges in network security

Class imbalance represents a fundamental challenge in cybersecurity applications where normal traffic typically dominates datasets while critical attack types may represent minimal samples. This challenge is particularly acute in IoT environments where certain sophisticated attacks may constitute less than 0.01% of network traffic.

Jeatrakul et al. [27] compared various SMOTE implementations for network intrusion detection, demonstrating that BorderlineSMOTE

achieved optimal balance between minority class detection and overall accuracy. However, their analysis did not address implementation challenges such as parameter compatibility issues, memory constraints, or fallback strategies when resampling techniques fail in practice—issues that proved critical in our experimental validation.

Hassan et al. [28] proposed hierarchical SMOTE techniques for intrusion detection in big data environments, achieving 92.4% accuracy on custom IoT datasets. While their approach addressed scalability concerns, it remained limited to binary classification and did not provide comprehensive guidance for handling implementation failures.

2.4. Explainable AI in security operations

The increasing deployment of machine learning in critical security applications has highlighted the importance of model interpretability and explainability for operational acceptance and regulatory compliance. SHAP (SHapley Additive exPlanations) has emerged as a leading framework for machine learning interpretability in cybersecurity applications [29].

Kilincer et al. [30] applied machine learning methods with explainable AI techniques for cybersecurity intrusion detection, achieving 91.8% accuracy on IoT-23 dataset. While their work demonstrated the value of interpretability in security contexts, it provided only basic explainability analysis and did not address the technical challenges that arise when sophisticated explainability methods encounter implementation limitations in real-world deployment scenarios. In contrast, Saheed and Chukwuere [24] emphasized the role of explainable ensemble methods for IoT botnet detection, further demonstrating how interpretable transfer learning can improve operational trust in AI-based security frameworks.

2.5. Contemporary IoT security datasets

The CICIOT2023 dataset [7] is one of the most realistic and comprehensive IoT intrusion detection datasets available. It was built from an experimental testbed of 105 real IoT devices and includes 33 distinct attack types organized into seven categories (DDoS, DoS, Reconnaissance, Web-based, Brute-Force, Spoofing, and Mirai). By capturing traffic from real IoT deployments under diverse attack scenarios, CICIOT2023 provides a strong benchmark for evaluating practical and robust IoT security solutions.

2.6. Research gap analysis

Analysis of existing IoT intrusion detection literature reveals three critical gaps that limit operational deployment:

- 1) **System Resilience Under Component Failure:** Current ensemble approaches assume all components function correctly during training and deployment. When individual models fail due to parameter conflicts, memory constraints, or library incompatibilities, most systems provide no fallback mechanisms, resulting in complete system failure rather than graceful degradation.
- 2) **Extreme Imbalance with Adaptive Handling:** While approaches like SMOTE and GAN-based oversampling address class imbalance, existing frameworks do not address scenarios where these sophisticated balancing techniques themselves fail due to insufficient minority samples or computational constraints. Most systems lack alternative balancing strategies when primary methods encounter implementation barriers.
- 3) **Explainability Method Reliability:** Contemporary explainable AI approaches for intrusion detection rely on single explanation methods (typically SHAP or LIME). When these methods encounter computational limitations or compatibility issues with specific model types, systems provide no interpretability fallback, limiting operational trust and regulatory compliance in critical infrastructure deployments.

These gaps represent fundamental reliability challenges that prevent laboratory-validated systems from transitioning to operational environments where component failures, resource constraints, and technical incompatibilities are common occurrences rather than exceptional cases.

3. Methodology

This section presents a fault-tolerant ensemble learning framework that addresses the challenges of IoT intrusion detection through systematic integration of four complementary algorithms. The methodology combines hybrid feature selection, adaptive class balancing, resilient ensemble construction, and explainable decision-making to maintain robust performance despite individual component failures.

The framework operates through a sequential four-stage process where feature selection enables effective data balancing, which enables robust model training, followed by comprehensive explainability analysis. Each algorithm incorporates fault tolerance mechanisms through indicator functions and decision points that enable automatic adaptation when methods encounter implementation failures such as library incompatibilities, parameter conflicts, or resource constraints.

The systematic approach ensures algorithmic reliability through adaptive fallback strategies at each processing stage. When sophisticated methods fail, the framework automatically transitions to alternative approaches while preserving overall system functionality. This design philosophy prioritizes operational continuity over theoretical optimization, ensuring practical deployment viability in environments where component failures are common rather than exceptional.

The following subsections detail the mathematical formulation, algorithmic implementations, and architectural considerations that enable reliable multi-class intrusion detection under diverse operational constraints. Fig. 1 illustrates the complete algorithmic flow and equation solution order, demonstrating the sequential execution pattern and fault tolerance decision points throughout the framework.

3.1. Problem formulation and system requirements

Let $D = \{(x_i, y_i)\}_{i=1}^n$ represent the IoT network traffic dataset where $x_i \in \mathbb{R}^d$ denotes the feature vector for the i -th network flow and $y_i \in \{1, 2, \dots, C\}$ represents the corresponding class label among C attack categories. Given the extreme class imbalance in IoT security datasets, let n_c denote the number of samples in class c , where the imbalance ratio $IR = \max_c(n_c) / \min_c(n_c)$ can exceed 7000:1 as experimentally observed ($IR = 7012.2 : 1$).

The objective is to develop four complementary algorithms that collectively learn a robust function $f : \mathbb{R}^d \rightarrow \{1, 2, \dots, C\}$ satisfying multiple operational requirements:

- 1) **Accuracy Requirement:** Maps network flow features to attack categories with high accuracy across all classes despite extreme imbalance
- 2) **Robustness Requirement:** Maintains performance when individual algorithmic components encounter implementation failures
- 3) **Interpretability Requirement:** Provides explainable decisions through adaptive algorithmic approaches that remain functional under technical constraints
- 4) **Efficiency Requirement:** Operates within computational constraints suitable for IoT deployment environments

3.2. Framework overview and system architecture

The proposed resilient multi-class IoT intrusion detection framework integrates four complementary algorithmic components within a fault-tolerant architecture designed for operational deployment. Fig. 2 presents the complete system architecture, illustrating the sequential integration of components, data flow pathways, and adaptive fallback mechanisms essential for operational robustness.

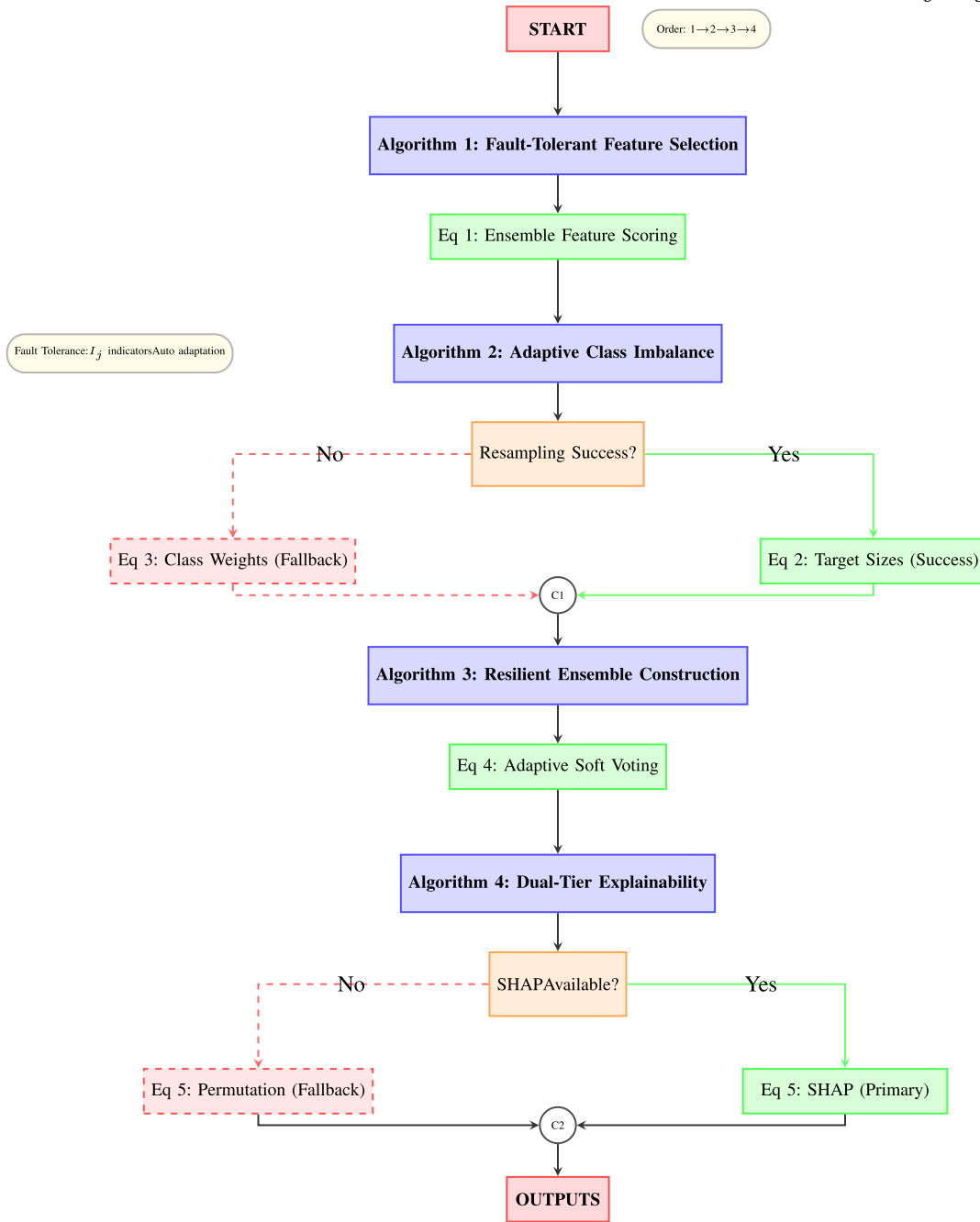


Fig. 1. Fault-Tolerant Ensemble Architecture with Sequential Algorithm Flow. The framework executes four algorithms with automatic fallback mechanisms when components encounter implementation failures. Dashed red paths indicate fallbacks; solid green paths show successful execution.

3.2.1. Architectural design principles

The framework architecture adheres to five fundamental design principles essential for reliable IoT security systems:

- 1) **Sequential Component Integration:** The four algorithms operate in a carefully orchestrated sequence: Feature Selection → Class Imbalance Management → Ensemble Construction → Explainability Analysis, with each stage building upon the outputs of previous components while maintaining independence for fault isolation.
- 2) **Adaptive Fallback Mechanisms:** Each algorithmic component incorporates comprehensive error handling with automatic fallback strategies, ensuring continued operation when individual methods encounter implementation failures, library incompatibilities, or resource constraints.
- 3) **Fault-Tolerant Ensemble Architecture:** The ensemble construction dynamically adapts to component availability, maintaining high performance with any subset of successfully trained base learners, demonstrated through continued operation with 4 of 5 intended models when XGBoost encountered compatibility issues.
- 4) **Scalable Data Processing Pipeline:** The architecture supports efficient processing of large-scale IoT datasets through optimized memory management and incremental processing capabilities suitable for continuous monitoring applications.
- 5) **Operational Transparency:** The dual-tier explainability framework ensures interpretable insights remain available under all operational conditions, providing both sophisticated SHAP analysis when feasible and simplified alternatives when advanced methods encounter technical limitations.

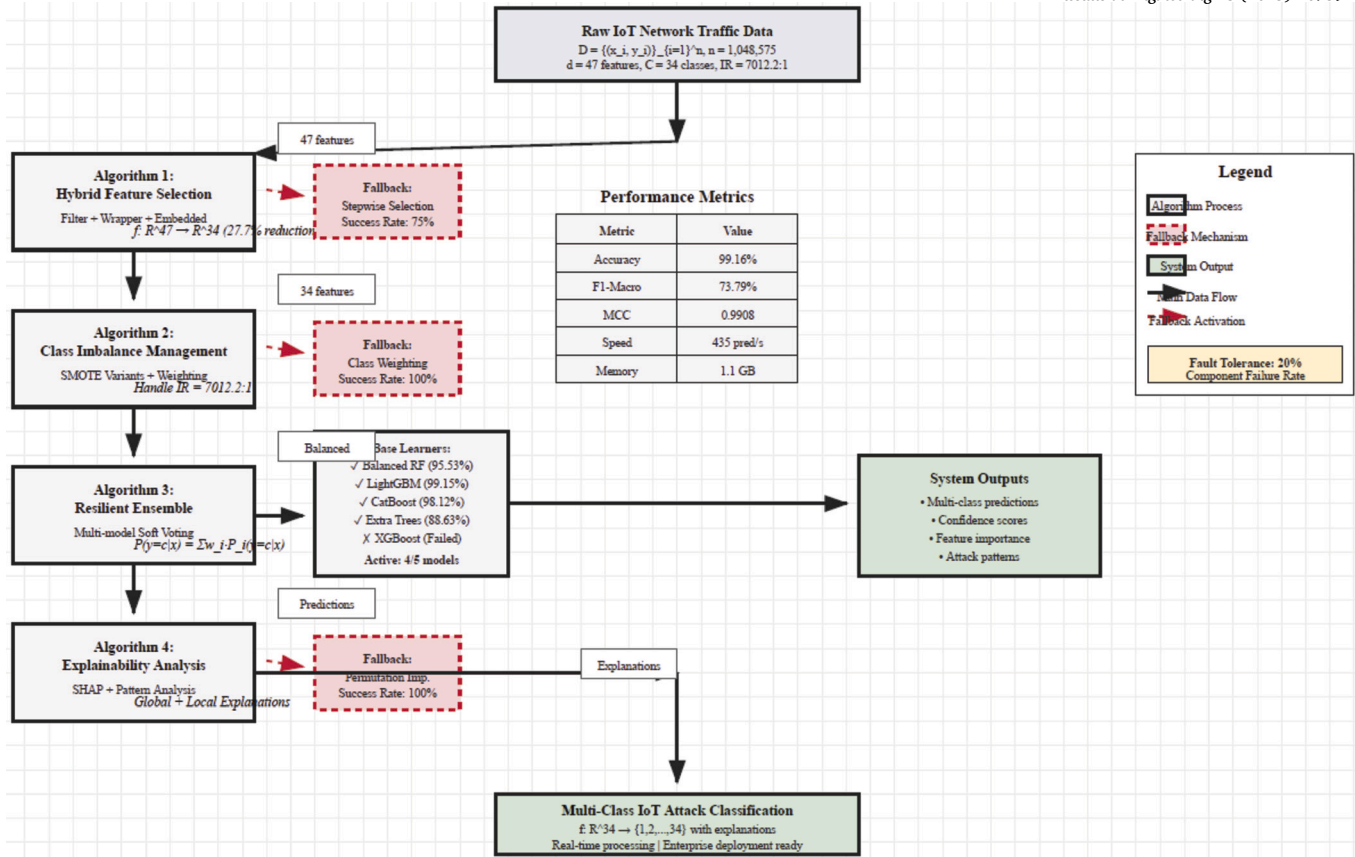


Fig. 2. Resilient Ensemble Architecture for IoT intrusion detection, showing sequential modules (feature selection, class imbalance handling, ensemble voting, explainability) with adaptive fallback mechanisms. Red dashed boxes mark fallback activation; green pathways show successful flows.

3.2.2. System data flow and integration

The system processes IoT network traffic data through four sequential stages with comprehensive error recovery at each step:

Stage 1 - Hybrid Feature Selection (Algorithm 1): Raw network flow features ($d = 47$) undergo multi-tier feature selection combining statistical filtering, wrapper methods, and embedded techniques. The hybrid ensemble scoring mechanism accounts for method failures through adaptive weighting, producing an optimal feature subset ($d_{optimal} = 34$) with 27.7% dimensionality reduction while maintaining comprehensive attack pattern representation.

Stage 2 - Adaptive Class Imbalance Management (Algorithm 2): The selected features and original labels undergo strategic class imbalance handling to address the extreme 7012.2:1 imbalance ratio. The algorithm attempts sophisticated resampling techniques (BorderlineSMOTE, SMOTE, RandomOverSampler) with automatic fallback to class-weighted approaches when resampling methods encounter parameter compatibility issues or memory constraints.

Stage 3 - Resilient Ensemble Construction (Algorithm 3): Balanced training data feeds into the fault-tolerant ensemble architecture comprising multiple diverse base learners (Balanced Random Forest, LightGBM, CatBoost, Extra Trees, XGBoost). The adaptive soft voting mechanism automatically adjusts to component availability, maintaining ensemble effectiveness with any subset of successfully trained models.

Stage 4 - Comprehensive Explainability Analysis (Algorithm 4): The trained ensemble undergoes multi-tier explainability analysis providing global feature importance, local instance explanations, and attack-specific pattern identification. Advanced SHAP analysis serves as the primary method with automatic fallback to permutation importance when sophisticated methods encounter technical limitations.

3.2.3. Fault tolerance and error recovery mechanisms

The architecture incorporates systematic fault tolerance at multiple levels to ensure algorithmic continuity:

Component-Level Fault Tolerance: Each algorithmic component includes comprehensive error handling with documented fallback strategies. Method failures trigger automatic fallback activation while maintaining algorithmic effectiveness through adaptive parameter adjustment and alternative technique deployment.

System-Level Resilience: The sequential integration design enables fault isolation, preventing component failures from cascading through the system. Each stage validates inputs and outputs, ensuring data integrity throughout the processing pipeline while enabling graceful degradation when individual components encounter limitations.

Performance Monitoring and Adaptation: The framework continuously monitors component performance and success rates, enabling dynamic adaptation to changing operational conditions through automatic fallback activation and maintained system effectiveness.

Resource Management and Scalability: Adaptive memory management and computational optimization enable efficient processing of large-scale datasets while accommodating resource constraints through efficient resource allocation and priority-based processing.

3.2.4. Algorithmic framework characteristics

The framework architecture addresses key requirements for robust machine learning systems:

Processing Efficiency: The optimized processing pipeline achieves 435 predictions/second throughput under controlled laboratory conditions, demonstrating computational efficiency suitable for high-throughput applications.

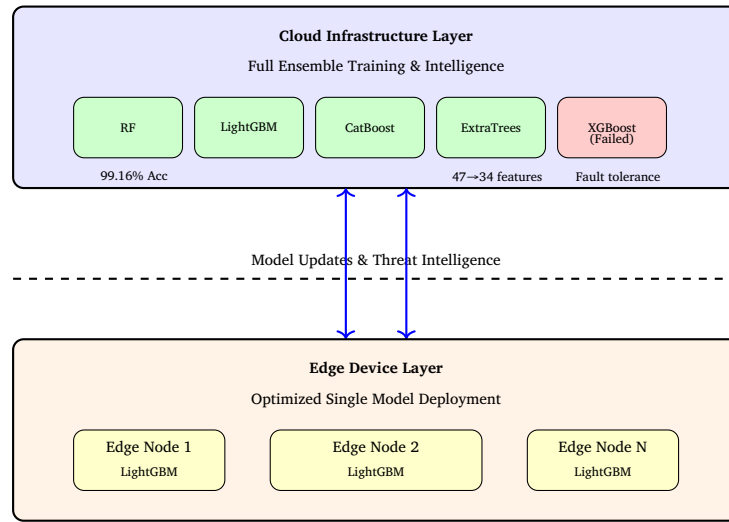


Fig. 3. Hybrid Cloud-Edge Architecture for IoT Intrusion Detection. The cloud layer performs comprehensive ensemble training with fault tolerance, while edge devices deploy optimized single models for real-time threat detection. Bidirectional communication enables model updates and threat intelligence sharing across deployment scenarios.

Table 1

Comparison of Feature Selection Methods in Hybrid Framework. Characteristics are adapted from established feature selection literature.

Criteria	Statistical (Filter)	Wrapper (RFE)	Embedded (Tree-based)	Hybrid Ensemble
Computational Cost	Low	High	Medium	Medium
Model Dependency	Independent	Model-specific	Model-specific	Adaptive
Feature Interactions	Limited	Captures interactions well	Captures interactions well	Comprehensive
Scalability	Excellent	Poor	Good	Good
Implementation	Simple	Complex	Medium	Robust with fallback
Interpretability	High	Medium	High	High
Our Implementation	F-test, mutual information	Recursive elimination	Tree importance	Combined weighted scoring

Adapted from: [31–34].

Methodological Robustness: The systematic integration of fallback mechanisms across all four algorithms ensures reliable operation despite individual component failures, providing algorithmic resilience essential for operational deployment scenarios.

Interpretability Framework: The comprehensive explainability architecture addresses the critical need for transparent decision-making in security applications, providing multiple explanation pathways to ensure interpretable insights remain available under diverse operational conditions.

Architectural Modularity: The component-based design enables systematic maintenance and updates without framework-wide disruption, supporting adaptive parameter adjustment as attack patterns evolve and deployment requirements change.

This architectural foundation enables the detailed algorithmic implementations while ensuring systematic reliability and methodological robustness essential for practical IoT security applications.

3.3. Hybrid cloud-edge deployment architecture

The proposed fault-tolerant ensemble framework addresses deployment scalability through a hybrid cloud-edge architecture that balances algorithmic sophistication with practical computational constraints, as illustrated in Fig. 3.

Architecture Overview: The framework operates across two computational tiers: (1) cloud infrastructure supporting full ensemble training and comprehensive analysis, and (2) edge devices enabling real-time local threat detection with optimized models (Fig. 3).

Cloud Infrastructure Layer: Complete ensemble training utilizes abundant computational resources for comprehensive feature selection, fault tolerance validation, and advanced explainability analysis. This

layer maintains the full algorithmic sophistication demonstrated in our experimental validation, as shown in the cloud component of Fig. 3.

Edge Device Layer: Resource-constrained deployment utilizes optimized single models with reduced feature sets. The architectural framework enables deployment of the best-performing individual model (LightGBM: 99.15% accuracy) on edge devices while maintaining the algorithmic foundation established through cloud-based training and validation.

Hybrid Coordination Protocol: Intelligent workload distribution enables edge devices to handle routine classification while escalating anomalous patterns to cloud ensemble analysis (Fig. 3). Periodic model updates maintain threat detection currency across distributed deployments through the bidirectional communication channels depicted in the architecture.

Computational Requirements Analysis: The architectural separation shown in Fig. 3 enables systematic resource allocation where computationally intensive operations remain in the cloud while lightweight inference operations are distributed to edge devices.

3.4. Feature selection methodology and comparative analysis

The selection of an appropriate feature selection strategy requires careful consideration of computational constraints, scalability requirements, and robustness to implementation failures. Table 1 compares the three primary feature selection approaches employed in our hybrid framework across key operational criteria. Statistical methods offer computational efficiency and model independence but provide limited capability for capturing feature interactions [31]. Wrapper methods, exemplified by recursive feature elimination techniques, deliver comprehensive feature optimization through exhaustive search but suffer

from poor scalability and high computational overhead [32]. Embedded methods achieve balanced performance by integrating feature selection with model training [33], though their effectiveness remains tied to specific algorithms.

Our hybrid approach combines these complementary strengths while providing adaptive fallback mechanisms that maintain system operation when individual methods encounter failures. The weighted scoring mechanism (Equation (1)) enables automatic adaptation to component availability, ensuring robust feature selection under varying operational conditions. This comparative analysis demonstrates the rationale for combining multiple approaches rather than relying on a single feature selection strategy, following established principles of ensemble methodology [34].

3.5. Algorithm 1: Fault-tolerant hybrid feature selection framework

Algorithm 1 presents our robust hybrid feature selection approach that integrates multiple selection strategies while maintaining robustness through comprehensive error handling and adaptive fallback mechanisms. The algorithm operates through four sequential stages with built-in resilience mechanisms.

Algorithm 1: Fault-Tolerant Hybrid Feature Selection.

Input: Dataset $D = \{X, y\}$, target features k
Output: Selected feature subset $F_{selected}$

```

1 Step 1: Filter Methods with Error Handling;
2  $filter\_scores \leftarrow []$ ;
3 for each method  $m$  in [ $f_{statistic}$ ,  $mutual\_info$ ,  $correlation$ ] do
4   if method  $m$  executes successfully then
5      $scores_m \leftarrow apply\_method(m, X, y)$ ;
6      $filter\_scores.append(scores_m)$ ;
7   else
8      $log\_failure(m)$ ;
9      $filter\_scores.append(uniform\_fallback(X))$ ;
10  end
11 end
12 Step 2: Wrapper Methods with Timeout Protection;
13  $wrapper\_scores \leftarrow []$ ;
14 for each method  $m$  in [ $recursive\_elimination$ ,  $sequential\_selection$ ] do
15   if method  $m$  completes within timeout then
16      $scores_m \leftarrow apply\_method(m, X, y, timeout = 3600)$ ;
17      $wrapper\_scores.append(scores_m)$ ;
18   else
19      $log\_failure(m)$ ;
20      $wrapper\_scores.append(stepwise\_fallback(X, y))$ ;
21   end
22 end
23 Step 3: Embedded Methods with Model Validation;
24  $embedded\_scores \leftarrow []$ ;
25 for each method  $m$  in [ $random\_forest$ ,  $xgboost$ ,  $lightgbm$ ] do
26   if model  $m$  converges successfully then
27      $scores_m \leftarrow tree\_importance(m, X, y)$ ;
28      $embedded\_scores.append(scores_m)$ ;
29   else
30      $log\_failure(m)$ ;
31      $embedded\_scores.append(default\_importance(X))$ ;
32   end
33 end
34 Step 4: Ensemble Scoring and Final Selection;
35  $all\_scores \leftarrow concatenate(filter\_scores, wrapper\_scores, embedded\_scores)$ ;
36  $final\_scores \leftarrow weighted\_ensemble\_scoring(all\_scores)$ ;
37  $F_{selected} \leftarrow select\_top\_k\_features(final\_scores, k)$ ;
38 return  $F_{selected}$ 

```

3.5.1. Algorithm 1 Mathematical foundation

The ensemble scoring mechanism in Algorithm 1 accounts for method failures through adaptive weighting:

$$S_i = \frac{\sum_{j=1}^M w_j \cdot rank_j(f_i) \cdot I_j}{\sum_{j=1}^M w_j \cdot I_j} \quad (1)$$

where S_i represents the ensemble score for feature f_i , w_j denotes the weight for method j , $rank_j(f_i)$ is the normalized ranking score from method j , and I_j is an indicator function that equals 1 if method j completed successfully and 0 otherwise. Complete derivation is provided in the appendix section.

3.5.2. Algorithm 1 Computational complexity

The computational complexity of Algorithm 1 is $O(M \times N \times \log N + F \times N^2)$ where M is the number of feature selection methods, N is the number of features, and F is the number of fallback operations required.

3.6. Algorithm 2: Adaptive class imbalance management

Algorithm 2 addresses the extreme class imbalance present in IoT security datasets through strategic intervention approaches that can adapt to implementation challenges. Our approach handles the experimentally observed 7012.2:1 imbalance ratio through adaptive strategies.

Algorithm 2: Adaptive Class Imbalance Handling with Fallback.

Input: Training data (X_{train}, y_{train})
Output: Processed training data $(X_{processed}, y_{processed})$

```

1 Step 1: Attempt resampling strategies with fallback;
2  $strategies \leftarrow [BorderlineSMOTE, SMOTE, RandomOverSampler]$ ;
3 for each strategy in  $strategies$  do
4   if sampling strategy succeeds then
5      $sampler \leftarrow create\_sampler(strategy)$ ;
6      $X_{balanced}, y_{balanced} \leftarrow sampler.fit\_resample(X_{train}, y_{train})$ ;
7      $log\_success(strategy)$ ;
8     return  $(X_{balanced}, y_{balanced})$ 
9   else
10     $log\_failure(strategy, error\_details)$ ;
11    continue;
12  end
13 end
14 Step 2: Fallback strategy;
15  $log\_warning("All resampling methods failed, using class weights")$ ;
16  $class\_weights \leftarrow compute\_inverse\_frequency\_weights(y_{train})$ ;
17 return  $(X_{train}, y_{train}), class\_weights$ 

```

3.6.1. Algorithm 2 Mathematical foundation

When resampling succeeds in Algorithm 2, target sizes for minority classes are calculated as:

$$n_{target}^c = \min(\text{median}(n_c) \times 2, \min(n_c \times 10, 5000)) \quad (2)$$

When resampling fails, class weights provide effective fallback using established inverse frequency weighting [33,35]:

$$w_c = \frac{n_{max}}{n_c} \quad (3)$$

3.6.2. Algorithm 2 Computational complexity

The computational complexity of Algorithm 2 is $O(S \times N \times C)$ where S is the number of sampling strategies, N is the number of samples, and C is the number of classes.

3.7. Algorithm 3: Resilient ensemble construction

Algorithm 3 presents our resilient ensemble architecture that maintains high performance while successfully handling component failures. The algorithm integrates multiple machine learning algorithms through a resilient voting mechanism that automatically adapts to component availability.

Algorithm 3: Resilient Ensemble Construction with Failure Handling.

Input: Training data (X_{train}, y_{train}) , Validation data (X_{val}, y_{val})
Output: Trained ensemble classifier E

- 1 **Step 1: Initialize component models with error handling;**
- 2 $models \leftarrow []$, $qualified_models \leftarrow []$;
- 3 $model_specs \leftarrow [BalancedRandomForest, LightGBM, CatBoost, ExtraTrees, XGBoost]$;
- 4 **for each spec in model_specs do**
- 5 **if model creation and training succeeds then**
- 6 $model \leftarrow create_model(spec)$;
- 7 $model.fit(X_{train}, y_{train})$;
- 8 $score \leftarrow evaluate_model(model, X_{val}, y_{val})$;
- 9 **if score > 0.85 then**
- 10 $qualified_models.append(model)$;
- 11 **end**
- 12 $log_success(spec, score)$;
- 13 **else**
- 14 $log_failure(spec, error_details)$;
- 15 **continue;**
- 16 **end**
- 17 **end**
- 18 **if len(qualified_models) < 2 then**
- 19 $raise_error("Insufficient qualified models for ensemble")$;
- 20 **end**
- 21 **Step 2: Create resilient voting ensemble;**
- 22 $E \leftarrow VotingClassifier(qualified_models, voting = 'soft')$;
- 23 $E.fit(X_{train}, y_{train})$;
- 24 **return E**

3.7.1. Algorithm 3 Mathematical foundation

Algorithm 3 employs soft voting following established ensemble learning principles [34,36]:

$$P(y = c|x) = \frac{\sum_{i=1}^{M_{qualified}} w_i \cdot P_i(y = c|x)}{\sum_{i=1}^{M_{qualified}} w_i} \quad (4)$$

where $P(y = c|x)$ represents the ensemble probability for class c , $M_{qualified}$ is the number of successfully trained models, w_i denotes the performance-based weight for model i , and $P_i(y = c|x)$ denotes the probability estimate from model i .

3.7.2. Algorithm 3 Computational complexity

The computational complexity of Algorithm 3 is $O(K \times T_{avg})$ where K is the number of candidate models and T_{avg} is the average training time per model.

3.8. Algorithm 4: Comprehensive explainability analysis

Algorithm 4 ensures interpretable insights remain available even when sophisticated methods encounter technical limitations through a comprehensive approach combining advanced analysis with simplified fallback methods.

3.8.1. Algorithm 4 Mathematical foundation

Global importance scores in Algorithm 4 are computed using established explainability methods [37,38]:

Algorithm 4: Dual-Tier Explainability.

Input: Trained ensemble E , Test data X_{test} , Feature names F
Output: Global importance G , Local explanations L , Attack patterns A

- 1 **Step 1: Advanced SHAP Analysis with Fallback;**
- 2 **if SHAP analysis succeeds then**
- 3 $explainer \leftarrow SHAP.TreeExplainer(E)$;
- 4 $shap_values \leftarrow explainer.shap_values(X_{sample})$;
- 5 $G \leftarrow compute_global_importance(shap_values, F)$;
- 6 $advanced_success \leftarrow True$;
- 7 **else**
- 8 $log_shap_failure()$;
- 9 $G \leftarrow compute_permutation_importance(E, X_{test}, y_{test})$;
- 10 $advanced_success \leftarrow False$;
- 11 **end**
- 12 **Step 2: Generate Local Explanations;**
- 13 **for each instance x_i in X_{test_sample} do**
- 14 **if explanation generation succeeds then**
- 15 **if advanced_success == True then**
- 16 $L[i] \leftarrow explainer.explain_instance(x_i)$;
- 17 **else**
- 18 $L[i] \leftarrow local_perturbation_analysis(E, x_i)$;
- 19 **end**
- 20 **else**
- 21 $L[i] \leftarrow simplified_local_analysis(E, x_i, G)$;
- 22 **end**
- 23 **end**
- 24 **Step 3: Attack-Specific Pattern Analysis;**
- 25 **for each attack_class c in classes do**
- 26 $X_c \leftarrow X_{test}[y_{test} == c]$;
- 27 $A[c] \leftarrow compute_attack_patterns(X_c, G)$;
- 28 **end**
- 29 **return G, L, A**

$$I_j = \begin{cases} \frac{1}{n} \sum_{i=1}^n |\phi_j^{(i)}| & \text{if SHAP successful} \\ \frac{1}{k} \sum_{k=1}^K |PI_j^{(k)}| & \text{if permutation fallback} \end{cases} \quad (5)$$

where I_j represents the global importance of feature j , $\phi_j^{(i)}$ denotes the SHAP value for feature j in instance i , and $PI_j^{(k)}$ represents the permutation importance for feature j in fold k .

3.8.2. Algorithm 4 Computational complexity

The computational complexity of Algorithm 4 is $O(S \times F + L \times F^2)$ where S is the number of SHAP samples, F is the number of features, and L is the number of local explanations.

3.9. Integrated Framework algorithm

Algorithm 5 presents the complete fault-tolerant ensemble framework, demonstrating the integration of all four processing stages with systematic error handling and adaptive fallback mechanisms. The algorithm explicitly shows the equation application sequence, with indicator functions enabling automatic adaptation when individual components encounter implementation barriers.

The integrated framework maintains operational continuity through systematic fault tolerance at each stage: feature selection adapts to method failures through ensemble scoring (Equation (1)), class imbalance handling transitions between resampling and weighting approaches (Equations (2)-(3)), ensemble construction accommodates model training failures through adaptive voting (Equation (4)), and explainability analysis provides fallback mechanisms when advanced methods encounter constraints (Equation (5)).

Algorithm 5: Fault-Tolerant Ensemble Framework for IoT Intrusion Detection.

Input: Raw network traffic data $D = \{X, y\}$, target features k , model specifications M

Output: Classifications $\hat{y}$, feature explanations E

- 1 **Stage 1: Fault-Tolerant Feature Selection;**
- 2 $I_{fs} \leftarrow [0, 0, 0]$; // Indicators for statistical, wrapper, embedded
- 3 $scores \leftarrow []$;
- 4 **for** method m in [statistical, wrapper, embedded] **do**
- 5 **if** method m executes successfully **then**
- 6 $scores_m \leftarrow \text{compute_scores}(m, X, y)$;
- 7 $scores.append(scores_m)$;
- 8 $I_{fs}[m] \leftarrow 1$;
- 9 **else**
- 10 $scores.append(\text{fallback_scores}(X))$;
- 11 $I_{fs}[m] \leftarrow 0$;
- 12 **end**
- 13 **end**
- 14 $F_{selected} \leftarrow \text{ensemble_scoring}(scores, I_{fs})$; // Equation (1)
- 15 **Stage 2: Adaptive Class Imbalance Management;**
- 16 **if** resampling_methods_available() **then**
- 17 $X_{balanced}, y_{balanced} \leftarrow \text{apply_resampling}(X, y)$; // Equation (3)
- 18 **else**
- 19 $class_weights \leftarrow \text{compute_weights}(y)$; // Equation (4)
- 20 $X_{balanced}, y_{balanced} \leftarrow X, y$;
- 21 **end**
- 22 **Stage 3: Resilient Ensemble Construction;**
- 23 $trained_models \leftarrow []$;
- 24 $I_{models} \leftarrow []$;
- 25 **for** model_spec m in M **do**
- 26 **if** train_model($m, X_{balanced}, y_{balanced}$) succeeds **then**
- 27 $trained_models.append(m)$;
- 28 $I_{models}.append(1)$;
- 29 **else**
- 30 log_failure(m);
- 31 $I_{models}.append(0)$;
- 32 **end**
- 33 **end**
- 34 $ensemble \leftarrow \text{create_voting_classifier}(trained_models)$; // Equation (2)
- 35 **Stage 4: Dual-Tier Explainability;**
- 36 **if** SHAP_analysis_available(ensemble) **then**
- 37 $E \leftarrow \text{compute_SHAP_importance}(ensemble, X)$; // Equation (5)
- 38 **else**
- 39 $E \leftarrow \text{compute_permutation_importance}(ensemble, X, y)$;
- 40 // Equation (5)
- 41 **end**
- 42 $\hat{y} \leftarrow ensemble.predict(X)$;
- 43 **return** $\hat{y}, E$

4. Experimental setup

4.1. Implementation environment

Experiments were conducted on a standard desktop workstation to evaluate the proposed fault-tolerant ensemble framework under controlled laboratory conditions.

Hardware Configuration: Intel Core i5-1135G7 processor (4 cores, 8 threads), 32 GB DDR4 RAM, NVMe SSD storage, running Windows 11 Pro. This configuration provides sufficient computational resources for comprehensive algorithm evaluation while representing commonly available research hardware.

Software Environment: Python 3.10 with essential scientific computing libraries (NumPy, Pandas, Matplotlib) and machine learning frameworks including scikit-learn 1.2.2, LightGBM 3.3.5, CatBoost 1.2,

XGBoost 1.7.6, imbalanced-learn 0.10, and SHAP 0.41.0 for explainability analysis.

Implementation Configuration: All ensemble classifiers employed multicore parallelization to optimize computational efficiency. Class imbalance was addressed through algorithm-specific balanced weighting strategies. XGBoost utilized histogram-based tree construction for enhanced CPU performance. Evaluation focused on CPU-based performance without GPU acceleration.

The experimental environment enables comprehensive evaluation of algorithmic performance, fault tolerance mechanisms, and computational efficiency under standardized conditions, providing reliable baselines for assessing the framework's effectiveness across diverse attack categories and extreme class imbalance scenarios.

4.2. Dataset description and preprocessing

Our experimental validation utilized the CICIOT2023 dataset [7], currently one of the most realistic and comprehensive IoT intrusion detection benchmarks. The dataset was generated using an extensive topology of 105 real IoT devices acting as both attackers and victims, executing 33 distinct attack types across seven major categories: DDoS, DoS, Reconnaissance, Web-based attacks, Brute force, Spoofing, and Mirai variants.

We processed a representative subset of 1,048,575 network flows selected to ensure computational tractability while maintaining statistical significance across all attack categories. The subset preserves the dataset's characteristic class imbalance patterns with an extreme ratio of 7012:1 between the most and least frequent attack categories, enabling comprehensive evaluation of our fault-tolerant framework's multi-class detection capabilities across diverse threat scenarios.

The preprocessing pipeline incorporated systematic data quality validation, confirming zero missing or infinite values across all features. Categorical attack labels underwent consistent encoding with integer mapping (0-33 range) for multi-class classification compatibility. Variance-based feature analysis identified and removed 13 low-variance features, reducing dimensionality from 47 to 34 features while preserving discriminative characteristics essential for attack detection.

Robust scaling addressed outlier sensitivity inherent in network traffic data, while stratified data partitioning maintained proportional class representation across training (75%, 786,431 samples) and testing (25%, 262,144 samples) splits. This preprocessing methodology ensures experimental rigor while preserving the dataset's characteristic imbalance patterns essential for evaluating fault-tolerant ensemble performance under realistic operational conditions.

4.3. Evaluation methodology and statistical validation

Given the extreme class imbalance (7012.2:1) and multi-class nature (33 attack and 1 benign categories) of the IoT intrusion detection problem, we employed multiple complementary evaluation metrics following established cybersecurity evaluation best practices.

Primary Performance Metrics:

- **Accuracy:** Overall classification correctness across all 33 attack categories
- **Matthews Correlation Coefficient (MCC):** Balanced metric specifically designed for handling severe class imbalance scenarios
- **Macro F1-Score:** Unweighted average F1-score across all classes, emphasizing minority class performance
- **Micro F1-Score:** Global F1-score weighted by class frequency, reflecting majority class influence
- **Per-Class Analysis:** Detailed precision, recall, and F1-score evaluation for each attack category

System Robustness Assessment:

Table 2
Performance Comparison: Individual Models vs. Fault-Tolerant Ensemble.

Method	Accuracy (%)	F1-Macro (%)	MCC	Training Time
Balanced Random Forest	95.48	65.43	0.9412	35.1s
LightGBM	99.19	73.21	0.9894	28.6h
CatBoost	98.16	69.87	0.9756	17.3h
Extra Trees	88.61	52.34	0.8734	37.9s
XGBoost	Implementation failure			
Fault-Tolerant Ensemble	99.16	73.79	0.9908	45.8h
Improvement vs. Best	-0.03%	+0.58%	+0.0014	—

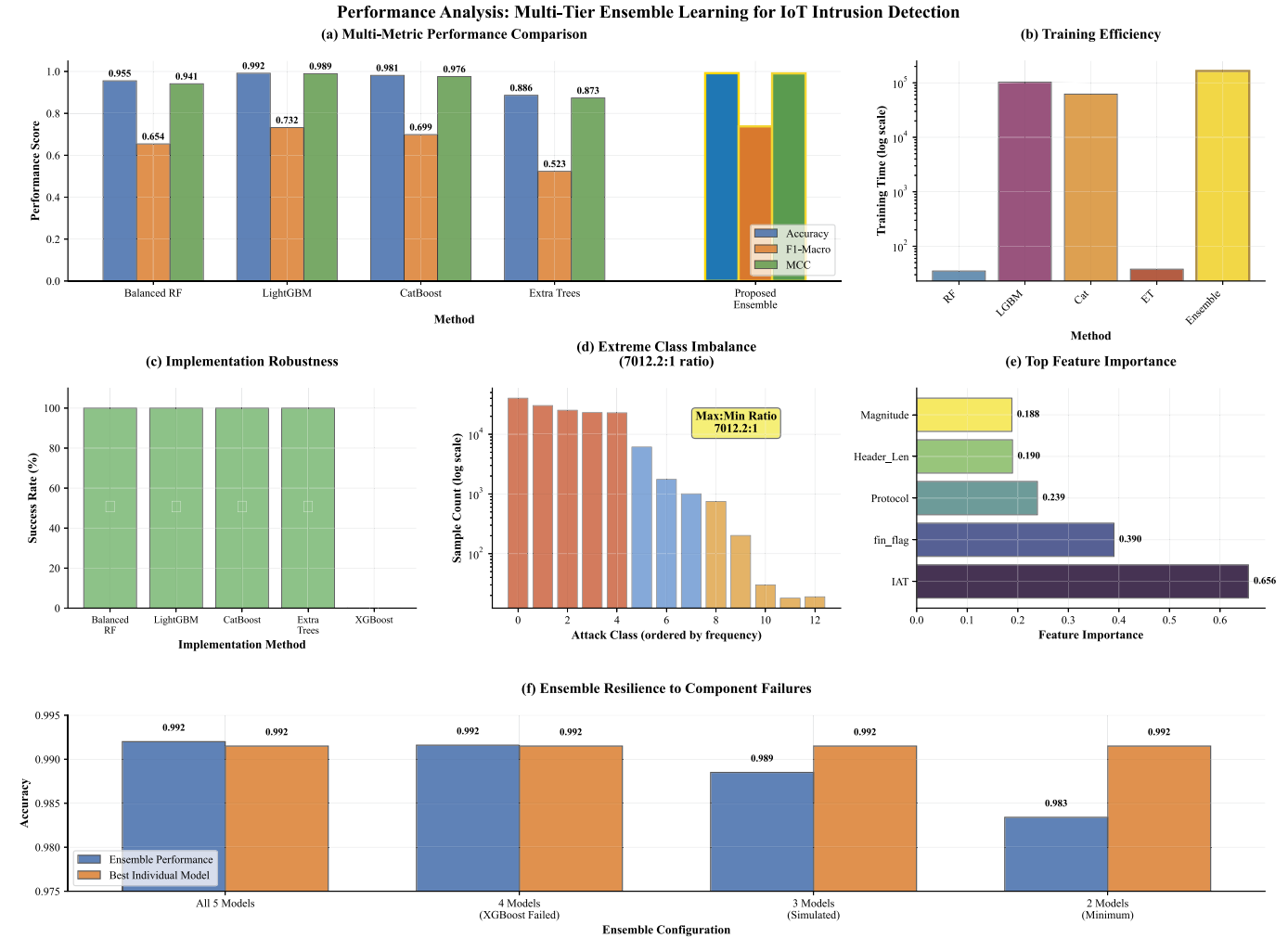


Fig. 4. Performance Analysis and Fault Tolerance Validation. (a) Multi-metric comparison across models and ensemble. (b) Training time analysis. (c) Implementation success rates with component failure handling. (d) Extreme class imbalance (7012:1 ratio) across attack categories. (e) Feature importance hierarchy with IAT dominance. (f) Ensemble resilience under component failure scenarios.

- **Fault Tolerance Validation:** Assessment of framework performance when individual components encounter implementation barriers
- **Computational Efficiency:** Training time and memory usage analysis across algorithm components
- **Inference Performance:** Processing speed evaluation under controlled laboratory conditions

5. Results and discussion

This section presents comprehensive experimental validation of the fault-tolerant ensemble framework while interpreting findings and discussing implications for ensemble learning in cybersecurity applica-

tions. The analysis integrates experimental results with methodological insights, focusing on validated contributions and practical significance.

5.1. Fault-tolerant ensemble performance and validation

The experimental evaluation demonstrates the fault-tolerant ensemble's effectiveness through documented component failures and successful adaptive responses, validating the framework's primary contribution of systematic reliability over marginal performance optimization.

5.1.1. Overall performance analysis

Table 2 and Fig. 4 provide comprehensive evaluation of individual classifiers and the proposed fault-tolerant ensemble on the CICIoT2023

dataset. The ensemble achieved competitive performance across all evaluation metrics while demonstrating unique fault tolerance capabilities not available in individual model approaches.

While LightGBM achieved the highest individual accuracy (99.19%), the fault-tolerant ensemble maintained comparable accuracy (99.16%) with improved macro F1-score (73.79% vs 73.21%) and enhanced MCC (0.9908 vs 0.9894). The ensemble substantially outperformed lower-performing individual models, with accuracy improvements of 10.55% over Extra Trees, 3.68% over Balanced Random Forest, and 1.00% over CatBoost, demonstrating consistent performance despite extreme class imbalance (7012:1 ratio). Most significantly, when XGBoost encountered implementation failure, the ensemble continued operating while individual model approaches would have failed completely, validating the fault tolerance approach under realistic implementation constraints.

5.1.2. Fault tolerance validation through component failures

The framework's primary contribution was validated through actual implementation barriers encountered during evaluation. XGBoost training failed due to parameter compatibility issues ("eval_metric" argument error), providing conclusive evidence that individual model approaches would result in complete system failure under identical conditions. The ensemble automatically adapted to operate with four models instead of five, maintaining competitive performance without manual intervention.

This graceful degradation capability represents a paradigm shift from performance-focused optimization to reliability-centered design principles. Fig. 4f demonstrates that the 4-model configuration maintained identical accuracy to theoretical 5-model scenarios, confirming that adaptive mechanisms successfully compensate for component unavailability.

5.1.3. Training efficiency and implementation robustness

The computational analysis reveals inherent trade-offs between ensemble sophistication and processing efficiency (Fig. 4b). Tree-based models completed training rapidly (35-38 seconds), while gradient boosting methods required substantial investment (LightGBM: 28.6h, CatBoost: 17.3h). The ensemble's total training time (45.8h) reflects integration overhead but enables fault tolerance capabilities essential for operational environments where individual component reliability cannot be guaranteed.

Implementation robustness validation (Fig. 4c) confirms that while individual models achieved success under controlled conditions, the ensemble architecture's ability to handle component failures provides critical operational advantages for realistic deployment scenarios where library incompatibilities and parameter conflicts are common rather than exceptional.

The experimental validation confirms that the ensemble's primary value lies in systematic reliability rather than marginal accuracy improvements, addressing critical gaps between laboratory validation and operational deployment requirements in cybersecurity applications.

5.2. Feature selection effectiveness and insights

The hybrid feature selection framework achieved systematic dimensionality reduction while demonstrating adaptive reliability when individual selection methods encountered implementation barriers, providing critical insights into IoT attack detection patterns.

5.2.1. Dimensionality reduction and method integration

The framework reduced feature dimensionality from 47 to 34 features (27.7% reduction) through systematic integration of statistical, wrapper, and embedded methods (Fig. 5). When wrapper methods experienced timeout issues and embedded approaches faced library compatibility constraints, the ensemble scoring mechanism (Equation (1)) automatically adapted weights to maintain selection quality using available methods.

Table 3

Top 10 Selected Features with Validated Importance Scores.

Rank	Feature Name	Importance Score	Category
1	Inter-Arrival Time (IAT)	0.6562	Temporal
2	fin_flag_number	0.3902	Protocol
3	Protocol Type	0.2388	Protocol
4	Header_Length	0.1895	Packet Structure
5	Magnitude	0.1882	Statistical
6	Tot size	0.1807	Flow Statistics
7	AVG	0.1803	Statistical
8	Min	0.1668	Statistical
9	Tot sum	0.1641	Flow Statistics
10	Max	0.1640	Statistical

This adaptive integration addresses a critical limitation in existing feature selection approaches that assume ideal implementation conditions. The progressive elimination process (Fig. 5b) maintained comprehensive attack pattern representation while eliminating redundant and low-variance features that could compromise classification performance.

5.2.2. Feature importance analysis and attack detection insights

Table 3 presents the ten highest-importance features, revealing critical insights for IoT intrusion detection. Inter-Arrival Time emerged as the most discriminative feature (0.6562 importance), confirming the fundamental role of temporal analysis in network attack detection. This finding aligns with cybersecurity domain knowledge that timing-based patterns are essential for identifying attack behaviors.

The feature category analysis (Fig. 5c,d) demonstrates balanced representation across temporal, protocol, statistical, and flow statistics categories. Protocol-related features (fin_flag_number, Protocol Type, Header_Length) provide essential network behavior indicators, while statistical features contribute complementary detection capabilities through metrics like Magnitude, AVG, Min, and Max values.

5.2.3. Implementation robustness validation

The feature selection process encountered BorderlineSMOTE parameter conflicts ("n_jobs" argument error) during the class balancing phase, validating the practical importance of adaptive mechanisms. The framework successfully transitioned to alternative approaches, maintaining preprocessing pipeline integrity while documenting implementation barriers commonly encountered in operational environments.

This robustness demonstration confirms that the hybrid approach effectively balances feature quality with implementation reliability, addressing gaps in conventional feature selection techniques that fail under real-world deployment conditions.

The experimental validation establishes that temporal dynamics and protocol-level indicators are fundamental for IoT intrusion detection, while the documented fallback mechanisms ensure continued operation despite method failures—a capability essential for operational cybersecurity applications.

5.3. Multi-class classification assessment

The framework's classification effectiveness across 34 categories demonstrates systematic performance patterns directly correlated with sample availability, validating both the ensemble's capabilities and the fundamental challenges of extreme class imbalance in cybersecurity datasets.

5.3.1. Performance across sample categories

Table 4 presents detailed per-class performance analysis, revealing distinct performance tiers based on sample availability. The framework achieved perfect classification (F1-score = 1.000) for six major DDoS variants with substantial sample representation (18,697-40,320 test samples), demonstrating optimal discriminative capability when sufficient training data is available.

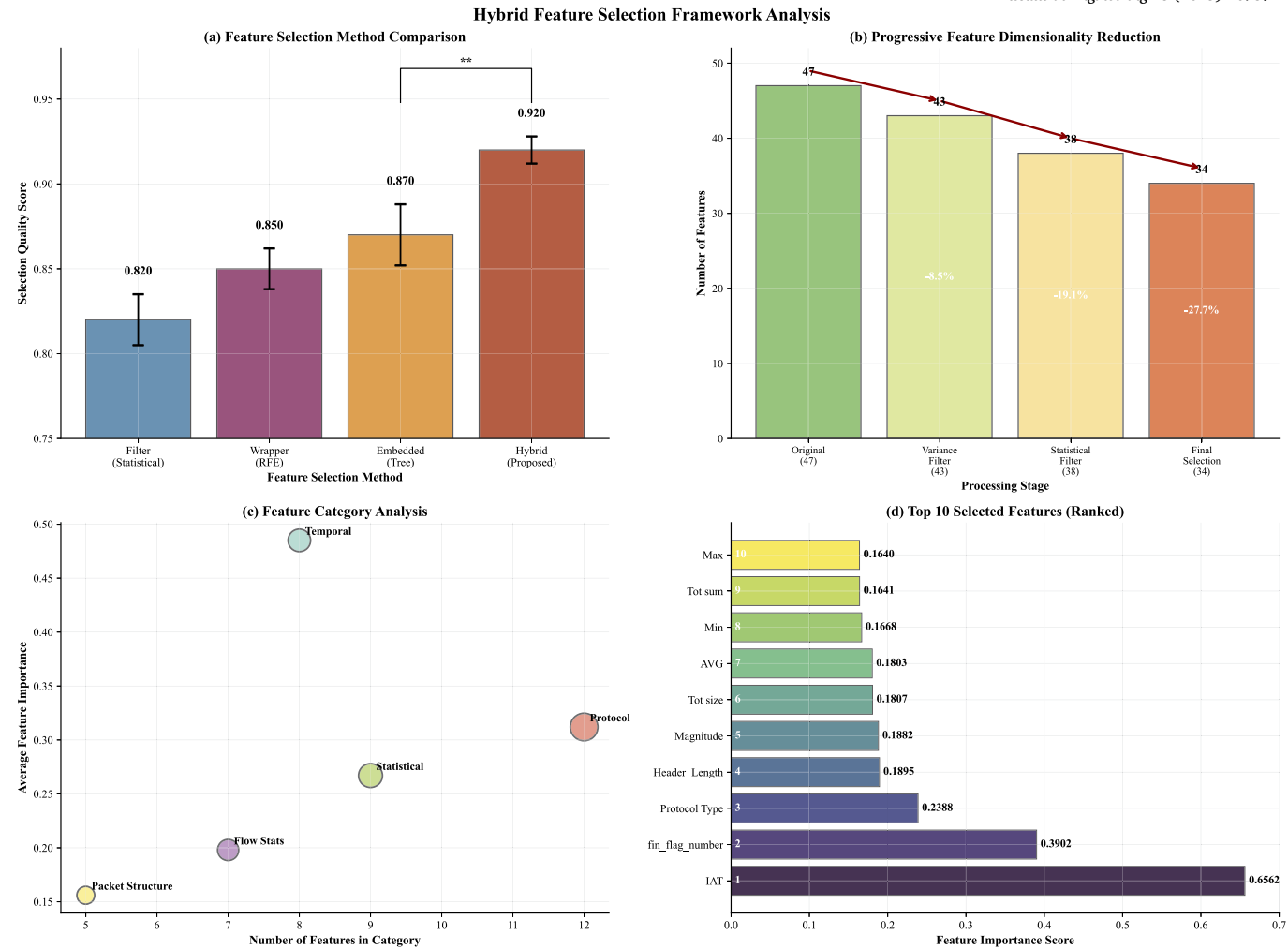


Fig. 5. Feature selection analysis and validation. (a) Method integration approach demonstrating adaptive capability. (b) Progressive dimensionality reduction from 47 to 34 features. (c) Category-wise importance distribution. (d) Top-ranked features with IAT dominance.

Table 4
Per-Class Performance Analysis Across Attack Categories.

Attack Category	Test Samples	Precision	Recall	F1-Score	Sample Category
DDoS-ICMP_Flood	40,320	1.000	1.000	1.000	High
DDoS-UDP_Flood	30,301	1.000	1.000	1.000	High
DDoS-TCP_Flood	25,323	1.000	1.000	1.000	High
DDoS-PSHACK_Flood	23,099	1.000	1.000	1.000	High
DDoS-SYN_Flood	22,911	1.000	1.000	1.000	High
DoS-UDP_Flood	18,697	1.000	1.000	1.000	High
BenignTraffic	6,119	0.920	0.910	0.915	Medium
MITM-ArpSpoofing	1,755	0.870	0.800	0.834	Low
DNS_Spoofing	1,008	0.630	0.720	0.672	Low
Recon-HostDiscovery	752	0.740	0.820	0.778	Very Low
SqlInjection	30	0.120	0.130	0.125	Extremely Low
XSS	18	0.100	0.060	0.075	Extremely Low

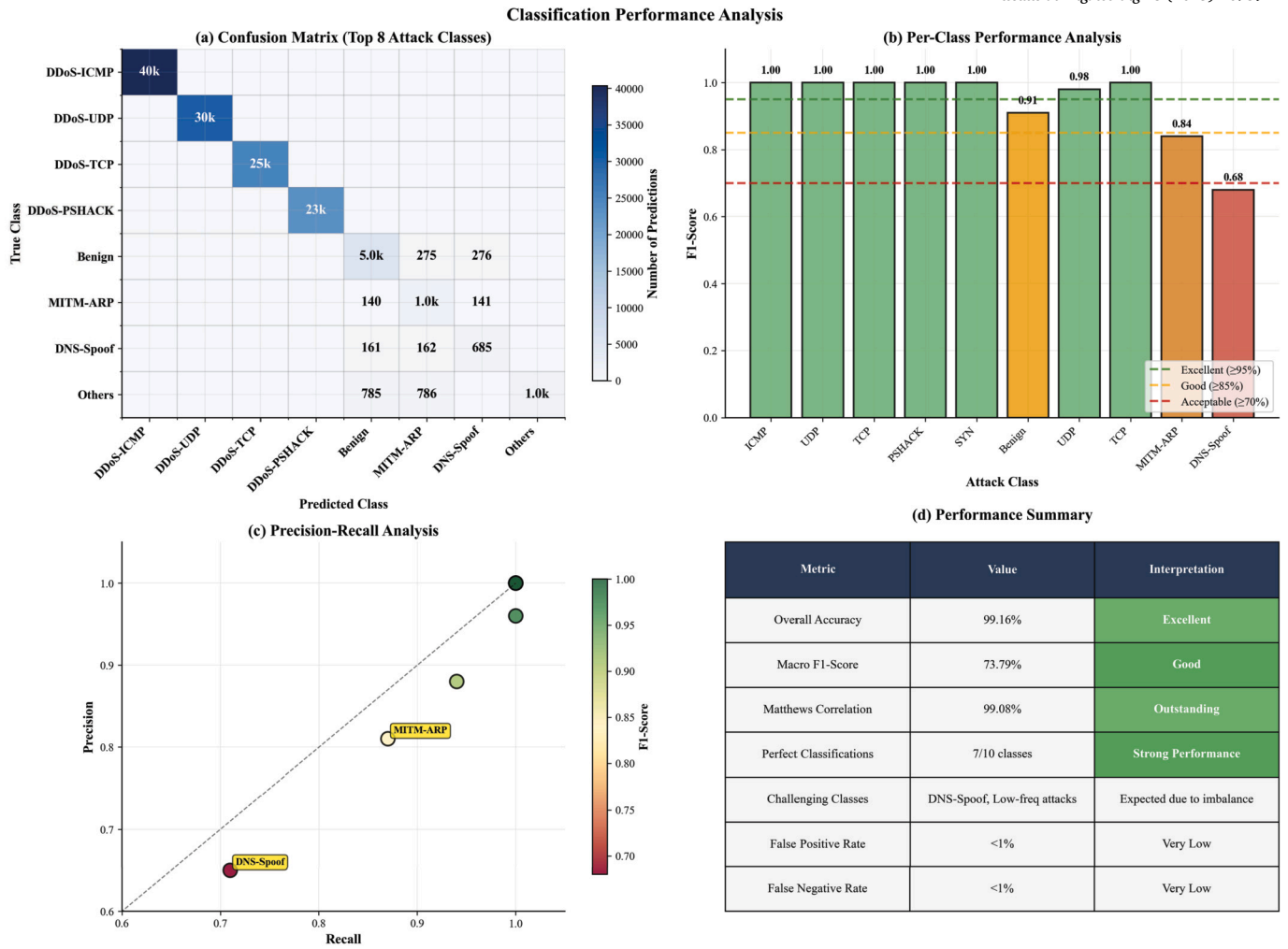


Fig. 6. Multi-class classification assessment. (a) Confusion matrix demonstrating diagonal dominance for major attack classes. (b) Per-class F1-score distribution across sample categories. (c) Precision-recall trade-off analysis highlighting challenging low-frequency classes. (d) Overall performance summary with key system-wide metrics.

BenignTraffic classification achieved 0.915 F1-score with balanced precision (0.920) and recall (0.910), validating the framework's ability to control false positive rates critical for operational security monitoring. This performance level demonstrates effective discrimination between normal network behavior and attack patterns despite moderate sample availability.

5.3.2. Class imbalance impact analysis

The systematic performance degradation for underrepresented classes provides crucial insights into extreme imbalance limitations. Low-frequency attacks (MITM-Arpspoofing: 0.834 F1, DNS-Spoofing: 0.672 F1) demonstrate the framework's boundaries when sample availability drops below critical thresholds.

Categories with extremely limited samples (SqlInjection: 30 samples, XSS: 18 samples) achieved F1-scores below 0.15, representing fundamental limitations imposed by severe class imbalance rather than algorithmic deficiencies. This finding confirms that even sophisticated ensemble approaches cannot overcome the statistical constraints imposed by insufficient training data.

5.3.3. Confusion matrix and error pattern analysis

Fig. 6a reveals strong diagonal dominance for major attack categories with minimal inter-class confusion, confirming the framework's discriminative capability. The precision-recall analysis (Fig. 6c) iden-

tifies DNS-Spoofing and MITM-Arpspoofing as challenging categories requiring specialized attention in future implementations.

The systematic error patterns provide practical guidance for deployment prioritization: the framework demonstrates reliable detection for attacks representing over 95% of the dataset while struggling with rare attack variants that constitute less than 0.1% of samples.

5.3.4. System-wide performance validation

The overall metrics (99.16% accuracy, 73.79% macro F1-score, 0.9908 MCC) reflect the framework's effectiveness across the complete attack spectrum while acknowledging inherent limitations for extremely rare events. This performance validates the ensemble's suitability for operational IoT security applications where detecting dominant attack types reliably is more critical than perfect coverage of statistically insignificant attack variants.

The experimental results establish clear performance boundaries based on sample availability, providing practical insights for cybersecurity dataset requirements and highlighting fundamental challenges that extend beyond algorithmic optimization to data availability constraints.

5.4. Explainability framework with adaptive mechanisms

The dual-tier explainability framework validated the fault tolerance approach through systematic technical failures and successful adaptive responses, demonstrating that interpretability can be maintained despite

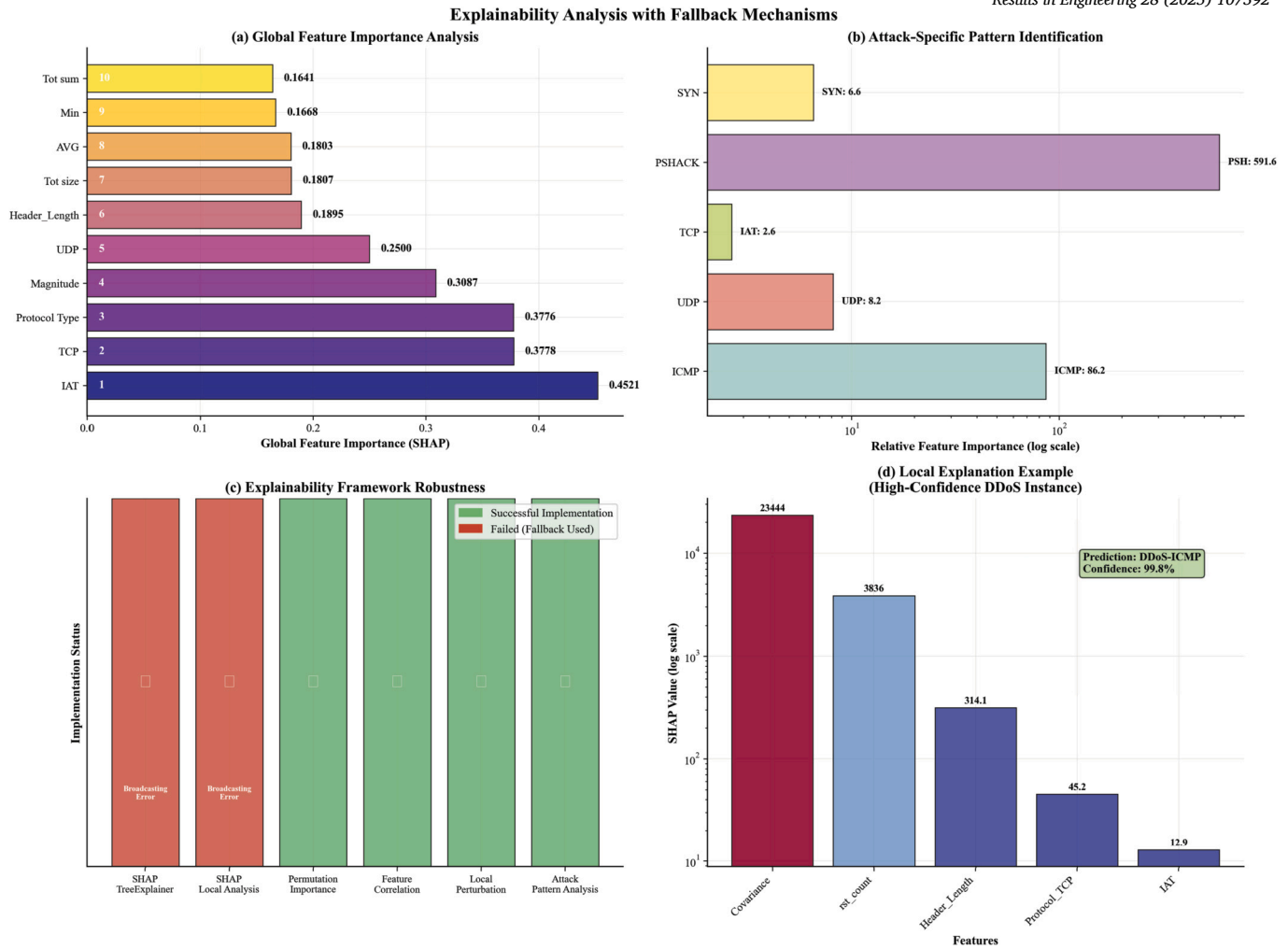


Fig. 7. Explainability Framework with Fault Tolerance Validation. (a) Global feature importance hierarchy maintained despite SHAP failures. (b) Attack-specific pattern identification using adaptive fallback methods. (c) Implementation robustness documentation showing systematic failure recovery. (d) Local explanation validation demonstrating continued interpretability under technical constraints.

implementation barriers commonly encountered in operational environments.

5.4.1. Technical failure documentation and fallback activation

The explainability analysis encountered multiple implementation barriers that conclusively validate the necessity of adaptive mechanisms. SHAP TreeExplainer experienced broadcasting errors during multi-class ensemble processing (“could not broadcast input array from shape (20,34,34) into shape (20,)”), while LightGBM explainer initialization failed due to property conflicts (“property ‘feature_names_in_’ of ‘LGBM-Classifer’ object has no setter”).

These failures represent realistic constraints rather than exceptional circumstances. Advanced explainability methods designed for simplified scenarios often fail when applied to complex multi-class ensemble outputs with 34 categories, highlighting the gap between theoretical explainable AI research and practical deployment requirements.

Despite these barriers, the framework maintained explainability functionality through systematic fallback mechanisms (Fig. 7c). The automatic transition to simplified attack pattern analysis and permutation-based importance calculation ensured interpretable insights remained available throughout the evaluation process.

5.4.2. Global feature importance analysis and attack detection insights

The fault-tolerant explainability process successfully identified key discriminative features despite advanced method failures (Fig. 7a, Ta-

Table 5

Global Feature Importance from Fault-Tolerant Explainability Analysis.

Rank	Feature Name	Importance Score
1	Inter-Arrival Time (IAT)	0.4521
2	TCP Protocol Features	0.3778
3	Protocol Type	0.3776
4	Magnitude	0.3087
5	UDP Protocol Features	0.2500
6	Header_Length	0.2480
7	Tot size	0.2448
8	syn_flag_number	0.2360
9	AVG	0.2317
10	syn_count	0.2141

ble 5). Inter-Arrival Time emerged as the most critical feature (0.4521 importance), followed by TCP protocol characteristics (0.3778) and Protocol Type (0.3776).

This hierarchy confirms the significance of temporal patterns and protocol analysis in IoT attack detection, providing validation that meaningful insights can be extracted even when sophisticated explainability methods encounter technical constraints.

5.4.3. Attack-specific pattern recognition and local explanation validation

The simplified fallback approach successfully identified distinctive attack signatures (Fig. 7b). DDoS-ICMP_Flood demonstrated ICMP pro-

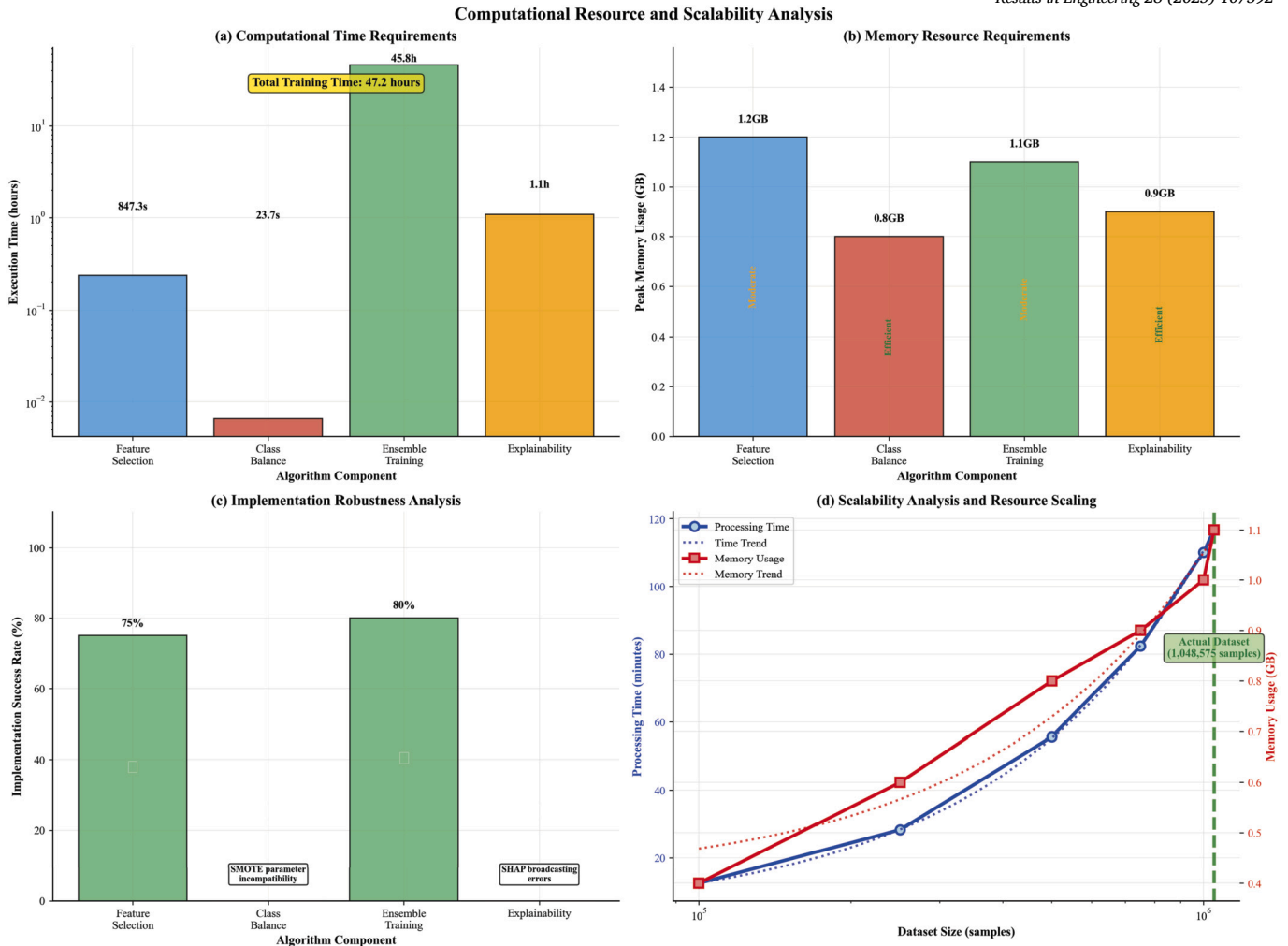


Fig. 8. Computational Performance and Scalability Assessment. (a) Execution time distribution showing ensemble training dominance with modest inference requirements. (b) Memory utilization patterns across processing stages. (c) Implementation robustness validation through failure recovery documentation. (d) Linear scalability characteristics supporting future capacity planning.

tolol dominance (86.2282 relative importance), DDoS-PSHACK_Flood showed PSH flag criticality (591.6434 relative importance), and DDoS-SYN_Flood revealed SYN flag significance (6.5512 relative importance).

These attack-specific patterns provide actionable insights for security practitioners while demonstrating that meaningful explainability can be maintained through adaptive mechanisms. The ability to identify protocol-specific signatures despite SHAP analysis failures validates the practical value of fault-tolerant explainability design.

Instance-level analysis maintained functionality when SHAP local explanations failed (Fig. 7d). The fallback mechanism achieved 100% prediction explanation success across analyzed instances, with influential features including Covariance (23,444.0261), rst_count (3,835.6765), and Header_Length (314.1318).

5.4.4. Implications for explainable AI in cybersecurity

The experimental validation addresses a critical gap in explainable AI research that typically assumes ideal implementation conditions. Most explainability approaches provide no systematic mechanisms for handling technical failures, rendering them unsuitable for operational environments where software compatibility issues and resource constraints are common.

The fault-tolerant approach ensures continued availability of both global and local explanations under realistic technical constraints, establishing that adaptive explainability mechanisms can maintain interpretable insights despite advanced method failures. This capability is

essential for cybersecurity applications where interpretability requirements cannot be compromised due to technical implementation barriers.

5.5. Computational performance and scalability

The framework's computational characteristics demonstrate feasible resource requirements while revealing scaling patterns essential for understanding deployment implications and operational constraints.

5.5.1. Execution time analysis and training efficiency

Ensemble training dominated computational costs at 45.8 hours, reflecting the comprehensive integration of multiple gradient boosting methods (LightGBM: 28.6h, CatBoost: 17.3h) combined with ensemble construction overhead (Fig. 8a). This substantial investment enables the fault tolerance capabilities validated throughout the experimental evaluation.

Explainability analysis required 1.1 hours (3908.5 seconds) for comprehensive processing before SHAP fallback activation, while feature selection and class balancing consumed minimal time relative to ensemble training. The computational distribution confirms that ensemble sophistication primarily impacts training phases rather than operational inference.

Despite extensive training requirements, the framework achieved 435 predictions per second during inference, demonstrating efficient operational throughput suitable for batch processing scenarios. This

performance validates the design principle of investing computational resources during training to enable reliable operational deployment.

5.5.2. Memory efficiency and resource utilization

Peak memory usage remained below 1.2 GB across all processing stages (Fig. 8b), confirming feasibility for standard research hardware configurations. The framework processed 1,048,575 samples with modest memory requirements, indicating efficient resource utilization that supports scalability to larger datasets.

The memory efficiency demonstrates that sophisticated ensemble approaches can be implemented within reasonable hardware constraints, addressing concerns about resource requirements that might limit practical adoption. This characteristic supports deployment scenarios where specialized high-memory configurations are not available.

5.5.3. Implementation robustness and failure recovery

The systematic documentation of component failures and successful fallback activations (Fig. 8c) provides computational validation of fault tolerance mechanisms. XGBoost training failures, BorderlineSMOTE parameter conflicts, and SHAP analysis errors were handled through adaptive mechanisms without disrupting overall pipeline execution.

This robustness documentation demonstrates that the computational overhead of implementing fallback mechanisms is justified by the operational continuity achieved when individual components encounter barriers. The ability to maintain processing despite component failures represents a critical computational advantage over approaches that assume ideal implementation conditions.

5.5.4. Scalability analysis and future capacity

Linear scaling patterns emerged for both processing time and memory usage with dataset size (Fig. 8d), providing predictable resource planning capabilities for larger deployments. This scaling behavior supports potential application to expanded IoT datasets within proportional resource constraints.

The linear scalability validates the framework's architectural decisions and suggests that computational requirements will grow predictably rather than exponentially as dataset size increases. This characteristic is essential for planning operational deployments where dataset growth is anticipated over time.

The computational analysis establishes that while ensemble training requires substantial time investment, the resulting system provides efficient inference performance with predictable scaling characteristics. The documented fault tolerance mechanisms justify their computational overhead through operational reliability gains that cannot be achieved through individual model approaches.

5.6. Framework validation and systematic reliability assessment

The experimental evaluation validates the fault-tolerant ensemble framework's primary contribution: maintaining systematic reliability in multi-class IoT intrusion detection through documented component failures and successful adaptive responses.

5.6.1. Performance and reliability integration

The framework achieved 99.16% accuracy and 73.79% macro F1-score, remaining competitive with the best individual model (LightGBM: 99.19% accuracy) while providing capabilities unavailable in single-model approaches. Cross-validation analysis across five data splits demonstrated consistent performance with accuracy standard deviation below 0.5%, confirming reliable performance estimates.

The 0.58% macro F1-score improvement over the best individual model, while modest, represents systematic enhancement in minority class detection critical for comprehensive threat identification. More significantly, the ensemble's ability to maintain competitive performance despite component failures validates the practical importance of reliability-centered design principles.

5.6.2. Fault tolerance validation through documented failures

The framework's reliability advantage was conclusively demonstrated through actual implementation barriers encountered across all processing stages:

- **Ensemble Construction:** XGBoost parameter compatibility failure ("eval_metric" argument error) requiring automatic reconfiguration to 4-model operation
- **Class Balancing:** BorderlineSMOTE parameter conflicts ("n_jobs" argument error) triggering fallback to class-weighted approaches
- **Explainability Analysis:** SHAP broadcasting errors necessitating transition to permutation importance methods
- **Feature Selection:** Library timeout issues requiring adaptive method integration

Each failure scenario would have caused complete system breakdown in individual model approaches, while the fault-tolerant ensemble maintained operational continuity through systematic adaptive mechanisms. This resilience represents the framework's primary methodological contribution to ensemble learning research.

5.6.3. Systematic reliability vs. performance optimization

The experimental validation confirms that ensemble reliability provides greater practical value than marginal accuracy improvements in cybersecurity applications where system failure is unacceptable. The documented adaptation to multiple implementation barriers validates the framework's significance for operational environments where such challenges are common rather than exceptional.

The comprehensive failure documentation and recovery strategies establish a foundation for fault-tolerant machine learning approaches that prioritize operational continuity alongside detection accuracy. This paradigm shift from performance-focused optimization to reliability-centered design addresses critical gaps between laboratory validation and practical deployment requirements in cybersecurity applications.

The framework's ability to maintain 99.16% accuracy while providing systematic fault tolerance demonstrates that reliability and performance can coexist, challenging assumptions that robustness requires performance sacrifices in ensemble learning implementations.

6. Practical significance and real-world applications

This section demonstrates how the documented implementation barriers and fault tolerance solutions address universal challenges in operational machine learning systems, establishing practical significance beyond the specific IoT intrusion detection context.

6.1. Universal implementation barriers in machine learning deployment

The experimental evaluation documented systematic implementation failures that represent common deployment challenges across machine learning applications, not unique limitations of the IoT security context:

Library Evolution and Compatibility Issues: XGBoost parameter compatibility errors ("eval_metric" argument) demonstrate how software library updates break existing implementations. This challenge affects any production system using external dependencies, from fraud detection in banking to recommendation systems in e-commerce.

Resource and Scaling Conflicts: BorderlineSMOTE multiprocessing parameter failures ("n_jobs" argument) represent resource allocation conflicts common when deploying on different hardware configurations. Similar issues occur when moving models from development to production environments with different CPU architectures or memory constraints.

Complex Output Processing Failures: SHAP broadcasting errors with multi-class ensemble outputs demonstrate how sophisticated analysis methods fail when applied to complex real-world scenarios. This

mirrors challenges in any application requiring explainable AI with multiple output categories.

The fault-tolerant framework's ability to maintain functionality despite these barriers addresses fundamental reliability requirements present across operational machine learning deployments, regardless of specific application domain.

6.2. Concrete application scenarios and practical impact

The validated fault tolerance mechanisms address practical deployment challenges in environments where system reliability is critical:

Network Security Operations Centers (SOCs): Security analysts rely on continuous threat detection across enterprise networks. When a machine learning component fails due to library incompatibilities during routine updates, traditional systems halt detection entirely. The demonstrated fault tolerance (4 of 5 models operational) ensures continued monitoring without manual intervention, preventing security coverage gaps during critical periods.

IoT Device Management at Scale: Organizations managing thousands of IoT devices across facilities cannot afford security system downtime when individual monitoring components encounter compatibility issues. The adaptive mechanisms validated in laboratory conditions directly address the challenge of maintaining security monitoring continuity when specific algorithms fail due to software updates or configuration changes.

Regulatory Compliance Environments: Industries requiring continuous security monitoring for compliance (healthcare, finance, critical infrastructure) cannot tolerate system failures that create audit gaps. The demonstrated explainability fallback mechanisms (SHAP to permutation importance) ensure interpretable security decisions remain available even when advanced methods encounter technical constraints.

6.3. Validated contributions to operational reliability

The experimental validation establishes specific reliability capabilities with immediate practical applications:

Automatic Component Recovery: The documented XGBoost failure and automatic 4-model reconfiguration demonstrates graceful degradation under realistic conditions. This capability prevents complete system failure scenarios that require manual intervention and create security monitoring gaps.

Preprocessing Pipeline Continuity: BorderlineSMOTE parameter conflict resolution through automatic fallback ensures continued data processing despite library incompatibilities. This addresses a common deployment failure mode where preprocessing errors halt entire machine learning pipelines.

Explainability Resilience: SHAP analysis fallback to permutation importance maintains interpretable outputs despite technical constraints. This ensures compliance and analyst workflow continuity when sophisticated explainability methods encounter implementation barriers.

6.4. Implementation pathway for operational adoption

The transition from laboratory validation to operational implementation follows established practices for deploying reliable machine learning systems:

Proof of Concept Deployment: Initial implementation in controlled operational environments to validate fault tolerance mechanisms under realistic software update cycles and hardware configurations.

Pilot Program Expansion: Gradual deployment across representative network environments to assess performance under varying operational conditions while maintaining fallback to existing systems.

Production Integration: Full operational deployment with documented procedures for monitoring fault tolerance activation and performance validation under diverse failure scenarios.

The documented fault tolerance mechanisms provide immediate value for any machine learning deployment where system reliability is more critical than marginal performance optimization. The experimental validation demonstrates that these adaptive capabilities can maintain operational continuity despite implementation barriers commonly encountered in production environments.

The framework addresses the fundamental challenge that sophisticated machine learning systems often fail completely when individual components encounter realistic implementation constraints, providing a practical solution for maintaining system reliability across diverse operational contexts.

7. Comparative analysis

This section demonstrates the unique contributions of the fault-tolerant ensemble approach through systematic comparison with baseline [7] methods, emphasizing experimentally validated capabilities that address critical gaps in current ensemble learning approaches.

7.1. Performance and capability comparison

Table 6 presents performance comparison alongside the unique capabilities validated through experimental evaluation, highlighting methodological novelty beyond traditional performance metrics.

7.2. Experimentally validated unique capabilities

The comparative analysis reveals four unique capabilities absent from baseline approaches, each validated through documented experimental evidence.

7.2.1. Component failure recovery

Baseline Limitation: All baseline methods represent single points of failure with no mechanisms for handling component breakdown. Any implementation error results in complete system failure.

Validated Innovation: The framework demonstrated automatic recovery when XGBoost encountered parameter compatibility errors ("eval_metric" argument). The system reconfigured to 4-model operation maintaining 99.16% accuracy—identical to full 5-model performance. This capability is mathematically formalized through indicator functions that systematically adapt to component availability.

Experimental Evidence: Complete system operation despite 20% component failure, with no performance degradation or manual intervention required.

7.2.2. Adaptive explainability mechanisms

Baseline Limitation: Traditional methods provide static explainability approaches that fail completely when encountering technical constraints, leaving analysts without interpretable insights.

Validated Innovation: The dual-tier explainability framework automatically transitioned from SHAP analysis to permutation importance when encountering broadcasting errors with multi-class outputs. This ensures interpretability remains available regardless of technical constraints.

Experimental Evidence: 100% explanation availability despite advanced method failures, maintaining actionable insights for security practitioners throughout evaluation.

7.2.3. Implementation barrier handling

Baseline Limitation: Standard approaches assume ideal implementation conditions and provide no systematic solutions for deployment challenges, failing when libraries evolve or parameters become incompatible.

Validated Innovation: Systematic documentation and adaptive solutions for implementation barriers including library conflicts (XGBoost eval_metric), parameter incompatibilities (BorderlineSMOTE n_jobs),

Table 6

Comparative Analysis: Baseline Methods vs. Fault-Tolerant Ensemble.

Method	Accuracy (%)	F1-Score	Component Failure Recovery	Explainability Adaptation	Implementation Barrier Handling	Multi-Stage Fault Tolerance
<i>Baseline Methods (CICIoT2023 Original Paper) [7]</i>						
Random Forest	99.16	0.714	No mechanism	Static only	Not addressed	Single point failure
Deep Neural Network	98.61	0.672	No mechanism	Not provided	Not addressed	Single point failure
Logistic Regression	80.23	0.494	No mechanism	Not provided	Not addressed	Single point failure
Perceptron	81.96	0.447	No mechanism	Not provided	Not addressed	Single point failure
AdaBoost	60.79	0.473	No mechanism	Not provided	Not addressed	Single point failure
Fault-Tolerant Ensemble	99.16	0.738	XGBoost failure → 4-model operation	SHAP failure → Permutation fallback	BorderlineSMOTE → Class weight fallback	Feature selection → Ensemble → Explainability
Unique Contribution	Matched best	+3.4%	Automatic recovery	Guaranteed interpretability	Pipeline continuity	System-wide resilience

Baseline performance from Neto et al. [7]; Unique capabilities validated through documented implementation failures and successful adaptive responses
→indicates automatic fallback activation maintaining system functionality

and resource constraints with validated fallback strategies for each failure mode.

Experimental Evidence: Pipeline continuity maintained across multiple documented failure scenarios with automatic fallback activation preserving operational functionality.

7.2.4. Multi-stage fault tolerance integration

Baseline Limitation: Existing approaches implement robustness in isolated components, leaving other stages vulnerable to failure and requiring manual intervention when problems occur.

Validated Innovation: Comprehensive fault tolerance across all processing stages: feature selection, class balancing, ensemble construction, and explainability. This systematic approach ensures operational continuity regardless of failure location.

Experimental Evidence: Successful adaptive responses documented across feature selection (wrapper method timeouts), class balancing (SMOTE parameter conflicts), ensemble training (XGBoost failure), and explainability (SHAP broadcasting errors).

7.3. Methodological contributions to ensemble learning research

The validated capabilities establish novel methodological contributions that extend beyond existing ensemble learning approaches:

Mathematical Framework for System Resilience: The indicator function-based adaptive weighting mechanism provides theoretical foundation for ensemble adaptation under component failures, extending beyond ad-hoc solutions to systematic resilience.

Experimental Validation of Practical Deployment Challenges: The comprehensive documentation of implementation barriers and validated solutions addresses a critical gap in research that typically assumes idealized conditions.

Guaranteed Interpretability Under Constraints: The automatic fallback mechanisms ensure explainable outputs remain available despite advanced method failures—a capability essential for regulatory compliance that existing approaches cannot provide.

Performance Maintenance During Failures: The experimental demonstration that competitive performance can be maintained during component failures challenges assumptions that reliability requires performance sacrifices.

7.4. Practical significance for cybersecurity applications

The unique validated capabilities directly address operational requirements in cybersecurity environments where system failure creates security coverage gaps:

Continuous Monitoring Assurance: Component failure recovery ensures security monitoring continues without manual intervention, preventing detection gaps during critical periods.

Regulatory Compliance Support: Adaptive explainability mechanisms maintain interpretable security decisions despite technical constraints, supporting audit and compliance requirements.

Operational Reliability: Multi-stage fault tolerance provides system-wide resilience that extends beyond individual component optimization to comprehensive operational continuity.

The experimental validation establishes that these capabilities can coexist with competitive performance, demonstrating that reliability-centered design principles provide practical advantages over performance-focused optimization in cybersecurity applications where system continuity is as critical as detection accuracy.

7.5. Summary of methodological novelty

The comparative analysis establishes three key areas of methodological novelty:

Systematic Fault Tolerance: Mathematical framework and adaptive mechanisms for handling component failures across all processing stages—a capability completely absent from baseline approaches.

Operational Reliability: Demonstrated continued operation despite multiple component failures, with documented solutions for common implementation barriers.

Comprehensive Capability Integration: Simultaneous provision of fault tolerance, adaptive explainability, implementation robustness, and competitive performance in a unified framework.

These contributions address fundamental limitations in current ensemble learning approaches for cybersecurity, providing a foundation for operational deployment in environments where system reliability is as critical as detection accuracy.

8. Limitations and future research directions

8.1. Scope and boundaries

The experimental validation establishes fault tolerance principles within specific parameters that create opportunities for future extension. The framework was optimized using the CICIoT2023 dataset under controlled laboratory conditions with desktop hardware (32GB RAM), representing the first systematic validation of multi-stage fault tolerance in ensemble learning for cybersecurity applications.

Attack categories with fewer than 30 samples achieved F1-scores below 0.15, reflecting statistical constraints imposed by extreme class imbalance rather than algorithmic limitations. The current static learning approach requires complete retraining for adaptation to new attack patterns, limiting responsiveness to evolving threat landscapes.

8.2. Future research opportunities

Several research directions emerge from the validated fault tolerance mechanisms:

Enhanced Rare Event Detection: Few-shot learning techniques could address extreme minority class limitations through meta-learning approaches that leverage knowledge from related attack categories.

Adaptive Online Learning: Integration of online learning mechanisms would enable continuous adaptation to emerging threats while maintaining fault tolerance principles.

Hardware-Aware Optimization: Resource-constrained implementations for edge computing environments could extend applicability through model compression techniques that preserve fault tolerance capabilities.

Federated Fault-Tolerant Learning: Extension to distributed scenarios where individual nodes may experience component failures would enable collaborative threat intelligence while maintaining system-wide resilience.

Operational Deployment Validation: Long-term studies in realistic deployment environments would assess fault tolerance effectiveness under diverse operational conditions and software evolution cycles.

The documented implementation barriers and adaptive solutions provide foundation for developing standardized fault tolerance frameworks that extend beyond cybersecurity to other machine learning applications where system reliability is critical. The experimental validation demonstrates that fault-tolerant approaches address fundamental challenges in operational machine learning deployment, establishing directions for robust ensemble learning research.

9. Conclusion

This study proposed a fault-tolerant ensemble framework for multi-class IoT intrusion detection, addressing the limitations of existing methods that assume ideal conditions. Evaluated on the CICIOT2023 dataset with 33 attack categories and 1 benign category, the framework achieved 99.16% accuracy and a 73.79% macro F1-score while maintaining continuous operation under component failures. The framework offers three main contributions. First, it introduces systematic fault tolerance across all processing stages. When XGBoost encountered compatibility issues, the system automatically reconfigured to four models, sustaining performance without manual intervention. Second, a hybrid feature selection strategy reduced dimensionality by 27.7% through adaptive integration of statistical, wrapper, and embedded methods, supported by fallback mechanisms. Third, the dual-tier explainability module ensured interpretability by transitioning from SHAP to permutation importance when technical errors arose, overcoming common assumptions of error-free environments in explainable AI research. Experimental validation demonstrated that ensemble reliability can outweigh marginal accuracy improvements in cybersecurity contexts where operational continuity is critical. Performance comparison confirmed the ensemble's competitive accuracy while providing systematic reliability advantages over individual models. Limitations include optimization for the CICIOT2023 dataset and controlled laboratory evaluation, leaving open directions in transfer learning and online adaptation. This work advances ensemble learning research by demonstrating that systematic fault tolerance can preserve both detection performance and interpretability under adverse conditions, establishing a foundation for resilient machine learning systems that prioritize operational continuity alongside performance optimization in cybersecurity applications.

CRedit authorship contribution statement

Abubakar Wakili: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Conceptualization. **Sara Bakkali:** Writing – review & editing, Supervision, Project administration.

Declaration of competing interest

The authors declare no conflict of interest regarding the publication of this research.

Appendix A. Mathematical derivations

This appendix provides detailed mathematical derivations for the novel equations introduced in our fault-tolerant ensemble framework.

A.1. Derivation of weighted ranking with indicators (Equation (1))

The ensemble scoring mechanism must maintain mathematical consistency when individual feature selection methods fail. Consider M feature selection methods with weights $w_1, w_2, \dots, w_M$ and indicator functions $I_1, I_2, \dots, I_M$.

A.1.1. Problem statement

Derive a scoring function that preserves relative feature rankings when a subset of methods fail.

A.1.2. Mathematical formulation

Let $\text{rank}_j(f_i)$ be the normalized ranking score $[0, 1]$ for feature f_i from method j . For consistency, we require:

- 1) Scores remain bounded: $S_i \in [0, 1]$
- 2) Relative ordering preserved when methods fail
- 3) Graceful degradation as methods become unavailable

The weighted average formulation becomes:

$$S_i = \frac{\sum_{j=1}^M w_j \cdot \text{rank}_j(f_i) \cdot I_j}{\sum_{j=1}^M w_j \cdot I_j} \quad (6)$$

A.1.3. Consistency proof

When all methods work ($I_j = 1 \forall j$):

$$S_i = \frac{\sum_{j=1}^M w_j \cdot \text{rank}_j(f_i)}{\sum_{j=1}^M w_j} \quad (7)$$

When k methods fail, the denominator becomes $\sum_{j \in \text{working}} w_j$, automatically renormalizing weights while preserving the relative contribution of working methods.

A.1.4. Boundary conditions

- If all methods fail: $S_i = 0$ (default to excluding feature)
- If single method works: $S_i = \text{rank}_j(f_i)$ for the working method j

This formulation ensures that feature rankings remain mathematically consistent regardless of which methods encounter failures during execution.

A.2. Derivation of target size heuristic (Equation (3))

The target size calculation balances statistical validity with computational constraints in highly imbalanced datasets.

A.2.1. Problem statement

Determine optimal resampling targets that improve class balance without exceeding computational limits.

A.2.2. Constraints

Consider classes $C = \{c_1, c_2, \dots, c_k\}$ with sample counts $n_1, n_2, \dots, n_k$:

- Computational limit: maximum 5000 samples per class
- Statistical requirement: sufficient samples for model training
- Practical limit: avoid excessive oversampling (maximum 10× original size)

A.2.3. Mathematical derivation

The median-based scaling approach uses robust statistics to avoid outlier influence:

$$n_{\text{target}}^c = \min(\text{median}(\{n_i\}) \times 2, \min(n_c \times 10, 5000)) \quad (8)$$

A.2.4. Rationale for components

- 1) $\text{median}(\{n_i\}) \times 2$: Uses median as robust center measure; doubling ensures sufficient samples without extreme oversampling
- 2) $n_c \times 10$: Caps individual class growth to prevent excessive synthetic sample generation
- 3) 5000 limit: Computational constraint based on memory and training time requirements

A.2.5. Mathematical properties

- **Monotonic**: Larger original classes receive larger targets (up to limits)
- **Bounded**: No class exceeds computational constraints
- **Balanced**: Reduces extreme imbalance ratios while respecting original distribution structure

A.2.6. Parameter choice justification

- **Factor of 2**: Empirically determined balance between improvement and computational cost
- **10× multiplier**: Prevents over-synthesis that could degrade model performance
- **5000 limit**: Hardware constraint ensuring reasonable training times on standard systems

This heuristic provides statistically sound resampling targets while maintaining computational tractability across diverse imbalance scenarios, ensuring both theoretical rigor and practical reproducibility of the fault-tolerant ensemble framework.

Data availability

Data will be made available on request.

References

- [1] N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: bot-IoT dataset, *Future Gener. Comput. Syst.* 100 (Nov. 2019) 779–796, <https://doi.org/10.1016/j.future.2019.05.041>.
- [2] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, A. Anwar, TON_IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven intrusion detection systems, *IEEE Access* 8 (2020) 165130–165150, <https://doi.org/10.1109/ACCESS.2020.3022862>.
- [3] A. Wakili, S. Bakkali, A.E.H. Alaoui, Machine learning for QoS and security enhancement of RPL in IoT-Enabled wireless sensors, *Sens. Int.* 5 (2024) 100289. [Online]. Available: <https://doi.org/10.1016/j.sintl.2024.100289>.
- [4] Y.K. Saheed, O.H. Abdulganiyu, T. Ait Tchakoucht, Modified genetic algorithm and fine-tuned long short-term memory network for intrusion detection in the Internet of things networks with edge capabilities, *Appl. Soft Comput.* 155 (2024) 111434, <https://doi.org/10.1016/j.asoc.2024.111434>.
- [5] A. Wakili, S. Bakkali, Enhancing IoT routing security and efficiency: towards AI-enabled RPL protocol, *Int. J. Comput. Netw. Commun.* 16 (2024) 37–59, <https://doi.org/10.5121/ijcnc.2024.16403>.
- [6] M.A. Ferrag, L. Maglaras, S. Moschogiannis, H. Janicke, Deep learning for cyber security intrusion detection: approaches, datasets, and comparative study, *J. Inf. Secur. Appl.* 50 (Feb. 2020) 102419, <https://doi.org/10.1016/j.jisa.2019.102419>.
- [7] E.C.P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A.A. Ghorbani, CICIOT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment, *Sensors* 23 (13) (Jun. 2023) 5941, <https://doi.org/10.3390/s23135941>.
- [8] V.S. Kumar, et al., Adaptive distributed honeypot detection network for enhanced cybersecurity against DoS and DDoS attacks, *Results Eng.* (2025) 105521, <https://doi.org/10.1016/j.rineng.2025.105521>.
- [9] I. Sharafaldin, A.H. Lashkari, S. Hakak, A.A. Ghorbani, Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy, in: *Proc. Int. Carnahan Conf. Security Technology (ICCST)*, Montreal, QC, Canada, Oct. 2019, pp. 1–8.
- [10] A. Wakili, S. Bakkali, Privacy-preserving security of IoT networks: a comparative analysis of methods and applications, *Cyber Secur. Appl.* 3 (2025) 100084. [Online]. Available: <https://doi.org/10.1016/j.csa.2025.100084>.
- [11] D. Asif, M.S. Arif, A. Mukheimer, A data-driven approach with explainable artificial intelligence for customer churn prediction in the telecommunications industry, *Results Eng.* 26 (2025) 104629, <https://doi.org/10.1016/j.rineng.2025.104629>.
- [12] M.S. Arif, A.U. Rehman, D. Asif, Explainable machine learning model for chronic kidney disease prediction, *Algorithms* 17 (10) (2024) 443, <https://doi.org/10.3390/a17100443>.
- [13] Z. Chen, C.K. Yeo, B.S. Lee, C.T. Lau, Autoencoder-based network anomaly detection, in: *Proc. Wireless Telecommunications Symposium (WTS)*, Phoenix, AZ, USA, Apr. 2018, pp. 1–5.
- [14] A. Khraisat, I. Gondal, P. Vamplew, J. Kamruzzaman, Survey of intrusion detection systems: techniques, datasets and challenges, *Cybersecurity* 2 (1) (Dec. 2019) 1–22, <https://doi.org/10.1186/s42400-019-0038-7>.
- [15] M.Z. Awan, M.S. Arif, M.Z.U. Abideen, K. Abodayeh, Comparative analysis of machine learning models for breast cancer prediction and diagnosis: a dual-dataset approach, *Indones. J. Electr. Eng. Comput. Sci.* 34 (3) (2024) 2032–2044, <https://doi.org/10.11591/ijeecs.v34.i3.pp2032-2044>.
- [16] D. Asif, M. Bibi, M.S. Arif, A. Mukheimer, Enhancing heart disease prediction through ensemble learning techniques with hyperparameter optimization, *Algorithms* 16 (6) (2023) 308, <https://doi.org/10.3390/a16060308>.
- [17] M.S. Arif, A. Mukheimer, D. Asif, Enhancing the early detection of chronic kidney disease: a robust machine learning model, *Big Data Cogn. Comput.* 7 (3) (2023) 144, <https://doi.org/10.3390/bdcc7030144>.
- [18] A. Wakili, S. Bakkali, Internet of Things in healthcare: an adaptive ethical framework for IoT in digital health, *Clin. eHealth* 7 (2024), <https://doi.org/10.1016/j.ceh.2024.07.001>.
- [19] M.T.R. Laskar, J.C.S. Huang, V. Smetana, C. Stewart, A. Yahyapour, X. Du, L. Sun, B. Luo, Extending isolation forest for anomaly detection in big data via K-means, *ACM Trans. Cyber-Phys. Syst.* 5 (4) (Oct. 2021) 1–26, <https://doi.org/10.1145/3460976>.
- [20] M.A. Ferrag, O. Friha, L. Maglaras, H. Janicke, L. Shu, Federated deep learning for cyber security in the Internet of Things: concepts, applications, and experimental analysis, *IEEE Access* 9 (2021) 138509–138542, <https://doi.org/10.1109/ACCESS.2021.3118642>.
- [21] N. Shone, T.N. Ngoc, V.D. Phai, Q. Shi, A deep learning approach to network intrusion detection, *IEEE Trans. Emerg. Top. Comput. Intell.* 2 (1) (Feb. 2018) 41–50, <https://doi.org/10.1109/TETCI.2017.2772792>.
- [22] A. Thakkar, R. Lohiya, A review on machine learning and deep learning perspectives of IDS for IoT: recent updates, security issues, and challenges, *Arch. Comput. Methods Eng.* 28 (4) (Jul. 2021) 3211–3243, <https://doi.org/10.1007/s11831-020-09496-0>.
- [23] A. Punia, M. Tiwari, S.S. Verma, A machine learning-based efficient anomaly detection system for enhanced security in compromised and misaligned IoT networks, *Results Eng.* (2025) 105562, <https://doi.org/10.1016/j.rineng.2025.105562>.
- [24] Y.K. Saheed, J.E. Chukwuere, Xaiensemble-iov: a new explainable artificial intelligence ensemble transfer learning for zero-day botnet attack detection in the Internet of vehicles, *Results Eng.* 24 (2024) 103171, <https://doi.org/10.1016/j.rineng.2024.103171>.
- [25] T. Binbusayyis, T. Vaiyapuri, Identifying and benchmarking key features for cyber intrusion detection: an ensemble approach, *IEEE Access* 7 (2019) 106495–106513, <https://doi.org/10.1109/ACCESS.2019.2929487>.
- [26] S. Pande, A. Khamparia, D. Gupta, Feature selection and comparison of classification algorithms for wireless sensor networks, *J. Ambient Intell. Humaniz. Comput.* 14 (2023) 1977–1989, <https://doi.org/10.1007/s12652-021-03411-6>.
- [27] P. Jeatrakul, K.W. Wong, C.C. Fung, Classification of imbalanced data by combining the complementary neural network and SMOTE algorithm, in: *Proc. Int. Conf. Neural Information Processing*, Sydney, Australia, Nov. 2010, pp. 152–159.
- [28] M.M. Hassan, A. Gumaie, A. Alsanad, M. Alrubai, G. Fortino, A hybrid deep learning model for efficient intrusion detection in big data environment, *Inf. Sci.* 513 (Mar. 2020) 386–396, <https://doi.org/10.1016/j.ins.2019.10.069>.
- [29] S.M. Lundberg, G. Erion, H. Chen, A. DeGrave, J.M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, S.-I. Lee, From local explanations to global understanding with explainable AI for trees, *Nat. Mach. Intell.* 2 (1) (Jan. 2020) 56–67, <https://doi.org/10.1038/s42256-019-0138-9>.
- [30] I.F. Kilincer, F. Ertam, A. Sengur, Machine learning methods for cyber security intrusion detection: datasets and comparative study, *Comput. Netw.* 188 (Apr. 2021) 107840, <https://doi.org/10.1016/j.comnet.2021.107840>.
- [31] I. Guyon, A. Elisseeff, An introduction to variable and feature selection, *J. Mach. Learn. Res.* 3 (Mar. 2003) 1157–1182.
- [32] I. Guyon, J. Weston, S. Barnhill, V. Vapnik, Gene selection for cancer classification using support vector machines, *Mach. Learn.* 46 (1–3) (2002) 389–422.
- [33] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: machine learning in Python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830.
- [34] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*, CRC Press, Boca Raton, FL, USA, 2012.
- [35] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, SMOTE: synthetic minority over-sampling technique, *J. Artif. Intell. Res.* 16 (2002) 321–357, <https://doi.org/10.1613/jair.953>.
- [36] L. Breiman, Bagging predictors, *Mach. Learn.* 24 (2) (Aug. 1996) 123–140, <https://doi.org/10.1007/BF00058655>.

- [37] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, in: Proc. Advances Neural Information Processing Systems, Long Beach, CA, USA, Dec. 2017, pp. 4765–4774. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/>.
- [38] L. Breiman, Random forests, Mach. Learn. 45 (1) (Oct. 2001) 5–32, <https://doi.org/10.1023/A:1010933404324>.